

Sistema de Predicción de Retrasos en las salidas y llegadas de los Vuelos

Máster de Formación Permanente en
Machine Learning e Inteligencia Artificial

Edición 2025/2026

Nombre y apellidos de los estudiantes:

- Juan Miguel Alvarez
- Riccardo Cast

Tutor del proyecto

- David Faustino

Tabla de contenido

1.	Resumen ejecutivo	4
2.	Problema que se quiere resolver.....	6
2.1.	Valor para los clientes	6
2.2.	Valor para las aerolíneas	6
3.	Solución tecnológica propuesta basada en IA.....	8
3.1.	Metodología	8
4.	Arquitectura propuesta incluyendo modelos/algoritmos a emplear.....	10
4.1.	Datos que serán utilizados en el proyecto	10
4.2.	Integración de fuentes	10
4.3.	Feature Engineering	16
4.4.	Modelos de Machine learning.....	19
4.5.	Estrategia de Entrenamiento.....	20
4.6.	Hiperparámetros Definidos	23
4.7.	Entrenamiento.....	25
4.8.	Pruebas extras con redes neuronales	31
4.9.	Tecnologías y entorno de desarrollo	32
5.	Simulación de costes (humanos y técnicos)	36
5.1.	Costes de Desarrollo.....	37
5.2.	Costes de implementación en producción	37
5.3.	Beneficios y ROI	38
5.4.	Plan de implementación.....	39
5.5.	Conclusiones.....	39
6.	Manual de usuario.....	40
6.1.	Objetivo	40
6.2.	Acceso al sistema.....	40
6.3.	Procedimiento de uso	40
6.4.	Funcionalidades adicionales.....	41
6.5.	Consideraciones	42
6.6.	Resolución de incidencias comunes	42
6.7.	Buenas practicas.....	42
6.8.	Soporte	42
7.	Conclusiones.....	43

8.	Próximos pasos.....	46
9.	Bibliografía.....	47

1. Resumen ejecutivo

El presente proyecto desarrolla un sistema para la predicción de retrasos en vuelos comerciales en Estados Unidos mediante técnicas de Machine Learning. A partir de un dataset de 6,7 millones de registros combinados con datos meteorológicos y operacionales, se construyó un pipeline integral de procesamiento, entrenamiento y evaluación de modelos.

El análisis comparativo permitió identificar un modelo con bajo error absoluto medio (MAE) y alta capacidad explicativa (R^2), manteniendo al mismo tiempo tiempos de inferencia compatibles con escenarios de aplicación en tiempo real.

Tras evaluar hasta 13 modelos de regresión y 4 redes neuronales, LightGBM se posicionó como la solución óptima, alcanzando:

- MAE de 5.0 minutos en retrasos de salida
- MAE de 3.72 minutos en retrasos de llegada
- R^2 superior a 0.97 en ambos casos

En comparación con la literatura académica y aplicaciones industriales reportadas en IEEE, Elsevier y publicaciones del Transportation Research Board, donde los modelos de regresión tradicionales como Random Forest o XGBoost presentan MAE promedio en el rango de 10 a 15 minutos y R^2 entre 0.90 y 0.94, la solución propuesta demuestra una mejora sustancial en precisión y capacidad explicativa.

El sistema incorpora una arquitectura modular y escalable, una interfaz predictiva en tiempo real desarrollada en Gradio y la generación automatizada de reportes explicativos mediante análisis SHAP, lo que facilita su interpretación y potencial adopción en entornos operativos reales.

Desde el punto de vista empresarial, la plataforma convierte la predicción de retrasos en una herramienta de gestión estratégica. No solo estima demoras con alta precisión, sino que identifica los factores críticos que las generan, transformando datos históricos en inteligencia accionable. Esto permite anticipar riesgos, optimizar la asignación de recursos, reducir congestiones y priorizar vuelos de mayor impacto económico. Asimismo, disminuye penalizaciones y costes operativos, mejora la eficiencia en el uso de combustible y tripulaciones, y permite decisiones preventivas sobre flota y mantenimiento, fortaleciendo la competitividad de la aerolínea.

Desde la perspectiva del cliente, la solución aporta mayor previsibilidad y transparencia en la planificación de viajes. Al anticipar posibles demoras, los pasajeros pueden reorganizar conexiones, traslados o compromisos profesionales con mayor seguridad, reduciendo la incertidumbre y mejorando la experiencia global de viaje.

Desde el punto de vista económico, el modelo presenta una elevada viabilidad con un ROI de 39.53% y un periodo de repago de 9 meses.

En conjunto, el proyecto demuestra la aplicabilidad real de la inteligencia artificial en la optimización operacional del sector aeronáutico, combinando precisión técnica, impacto económico y generación de valor tanto para la aerolínea como para el pasajero.

Nota:

En trabajos publicados en: IEEE – Elsevier - Transportation Research Board

Lo más común es reportar:

- MAE
- RMSE
- R^2

En investigaciones académicas y sistemas reales de aerolíneas / aeropuertos, lo más común es:

MAE (Mean Absolute Error) → La más usada

Por qué se usa:

- Se interpreta directamente en minutos
- No explota con ceros
- Es robusta y estable
- Fácil de comunicar a operaciones

RMSE (Root Mean Squared Error)

Por qué se usa:

- Penaliza más los errores grandes
- Útil cuando los delays extremos son costosos
- En papers de aviación suele reportarse junto con MAE.

R^2 Se usa principalmente en papers académicos.

2. Problema que se quiere resolver

Hoy en día las empresas de transporte aéreo utilizan sistemas de gestión operativa que permiten monitorear información estructurada como origen, destino, horario programado de salida y llegada, aeronave asignada y cantidad de pasajeros.

Sin embargo, estos sistemas funcionan principalmente de manera descriptiva y reactiva, ya que no incorporan modelos predictivos capaces de anticipar retrasos en función de múltiples variables internas y externas.

Los retrasos en vuelos pueden estar influenciados por factores como:

- Condiciones meteorológicas
- Congestión aeroportuaria
- Hora del día
- Día de la semana o estacionalidad
- Distancia del vuelo
- Historial de puntualidad de la aeronave
- Efecto cascado de vuelos previos
- Eventos extraordinarios

Actualmente, la toma de decisiones ocurre cuando el retraso ya se ha materializado, lo que genera:

- Incremento de costes operativos (combustible, reprogramación, horas extra de tripulación)
- Penalizaciones económicas y compensaciones a pasajeros
- Deterioro en indicadores de puntualidad.
- Impacto reputacional y pérdida de fidelización
- Permite organizar con más precisión reuniones, lugar a visitar, transporte a usar, hoteles

Desde el punto de vista teórico, el problema puede formularse como:

- Un problema de clasificación binaria:
 - Predecir si un vuelo se retrasará (Sí/No).
- Un problema de clasificación multiclase:
 - Clasificar el retraso en categorías (0–15 min, 15–60 min, >60 min).
- Un problema de regresión:
 - Estimar la cantidad exacta de minutos de retraso.

2.1. Valor para los clientes

Desde la perspectiva del pasajero, la solución permitirá:

- Obtener una estimación anticipada del horario real de llegada, facilitando la planificación de viajes personales o de negocios.
- Identificar trayectos con mayor propensión a retrasos.
- Evaluar el desempeño histórico de las aerolíneas en términos de puntualidad, facilitando una elección basada en evidencia y no únicamente en precio u horario.

2.2. Valor para las aerolíneas

Desde el punto de vista empresarial y operativo, la plataforma ofrece múltiples beneficios estratégicos:

2.2.1. Identificación de factores críticos

El modelo permite analizar qué variables influyen significativamente en los retrasos, tales como:

- Aeropuerto de origen o destino
- Condiciones meteorológicas
- Fabricante y modelo de la aeronave
- Antigüedad del avión
- Trayectoria

Este análisis contribuye a comprender patrones operativos y tomar decisiones basadas en evidencia.

2.2.2.Optimización operativa

- Mejor planificación de recursos (puertas de embarque, tripulaciones y personal).
- Reducción de cuellos de botella y congestión en tierra o en el espacio aéreo.
- Priorización de vuelos críticos o con conexiones estratégicas.

2.2.3.Reducción de costes

- Disminución de penalizaciones y compensaciones por demoras.
- Ahorro en combustible al evitar tiempos innecesarios de espera en pista o taxi.
- Mejor gestión de jornadas laborales de tripulación, reduciendo horas extra no planificadas.

2.2.4.Planificación estratégica y mantenimiento

- Posibilidad de programar mantenimientos preventivos o ajustar rotaciones cuando se detecten patrones asociados a mayores probabilidades de retraso.
- Toma de decisiones anticipada mediante la redistribución de flota o personal antes de que el retraso se materialice.

En conjunto, la plataforma incorpora una capa predictiva a los sistemas de gestión actuales, permitiendo transformar datos históricos y variables operativas en información accionable.

3. Solución tecnológica propuesta basada en IA

Con el objetivo de anticipar retrasos en vuelos y transformar la toma de decisiones de reactiva a proactiva, se propone el desarrollo de un sistema predictivo basado en técnicas de Inteligencia Artificial y Machine Learning.

La solución consiste en la implementación de un modelo de regresión supervisada capaz de estimar la cantidad de minutos de retraso de un vuelo, tanto en su despegue como en su aterrizaje, utilizando datos históricos y variables operativas, temporales y externas. El sistema será desarrollado en Python y contará con una interfaz gráfica interactiva desplegada en Gradio.

Se opta por un enfoque de regresión supervisada debido a que la variable objetivo corresponde a una magnitud continua (minutos de retraso). Este enfoque permite obtener estimaciones más precisas y accionables desde el punto de vista operativo, al cuantificar la magnitud del impacto y facilitar la priorización de recursos.

Además, se elige un modelo de regresión frente a uno de clasificación, ya que proporciona información más detallada al estimar el valor exacto del retraso. De esta manera, se evita la pérdida de información que implica discretizar la variable en categorías previamente definidas. Además, la categorización requiere establecer umbrales arbitrarios (por ejemplo, 15 o 30 minutos), los cuales pueden variar según el criterio operativo o el contexto de aplicación.

El modelo permite analizar qué variables influyen significativamente en los retrasos, tales como:

- Aeropuerto de origen y destino
- Condiciones meteorológicas previstas
- Fabricante y modelo de la aeronave
- Antigüedad del avión
- Características del trayecto y duración del vuelo
- Tráfico aéreo

A partir del análisis de importancia de variables, la aerolínea podrá identificar patrones estructurales que influyen en la puntualidad y diseñar estrategias de mitigación.

3.1. Metodología

3.1.1. Comprensión del negocio

Se define como objetivo principal la predicción anticipada de retrasos en vuelos, permitiendo mejorar la planificación operativa y reducir costes asociados a demoras.

3.1.2. Comprensión y preparación de los datos

Se realiza la recolección, integración y limpieza de datos históricos de operaciones aéreas. Las tareas incluyen:

- Tratamiento de valores nulos
- Normalización y transformación de variables
- Eliminación de inconsistencias
- Generación de nuevas variables derivadas

3.1.3. Análisis exploratorio de datos (EDA)

Se estudian patrones, tendencias temporales y relaciones entre variables explicativas y la variable objetivo (minutos de retraso), identificando posibles correlaciones y distribuciones relevantes.

3.1.4. Modelado

Se evalúan distintos algoritmos de regresión supervisada y una opción de red neuronal, realizando una división de datos en conjuntos de entrenamiento y prueba.

3.1.5. Evaluación

Se miden métricas como RMSE, MAE, tiempo de predicción y R^2 , comparando el desempeño de los modelos y seleccionando el más adecuado según criterios de precisión, performance e interpretabilidad.

3.1.6. Despliegue y monitoreo

El modelo seleccionado se integra en gradio, permitiendo generar predicciones en tiempo real y facilitando el monitoreo continuo de su desempeño.

En esta sección se presenta el enfoque metodológico y los componentes principales de la solución propuesta. El desarrollo técnico detallado, incluyendo la arquitectura implementada, los modelos evaluados y los resultados obtenidos, se expone en profundidad en el Apartado 4.

4. Arquitectura propuesta incluyendo modelos/algoritmos a emplear

4.1. Datos que serán utilizados en el proyecto

Los datos utilizados para decidir cuál es el mejor modelo, son los datos históricos de los registros de los retrasos que las aerolíneas han tenido.

En este ejercicio hemos tomado los datos históricos de varias aerolíneas durante el 2023.

La base de datos utilizada proviene del dataset 2023 US Civil Flights Delay, Meteo and Aircraft, disponible en la plataforma Kaggle [1].

Este dataset integra información operacional, meteorológica y técnica de las aeronaves, permitiendo un análisis multifactorial de los retrasos.

4.2. Integración de fuentes

Para construir una base consolidada adecuada al modelo predictivo, se realizó un proceso de integración de diferentes conjuntos de datos:

- Dataset principal de vuelos (información operacional y retrasos)
- Dataset de aeropuertos (nombres, ubicación geográfica, coordenadas)
- Dataset meteorológico (temperatura, precipitación, viento, presión, nieve)

Esto permitió generar una base unificada que contiene:

- Variables temporales
- Variables operativas
- Variables meteorológicas
- Variables técnicas de la aeronave
- Variables geográficas

A continuación, se presenta el conjunto de variables consideradas para el análisis, correspondiente al dataset consolidado luego del proceso de integración de fuentes. Estas variables constituyen la base sobre la cual se desarrolla el análisis exploratorio y el posterior modelado.

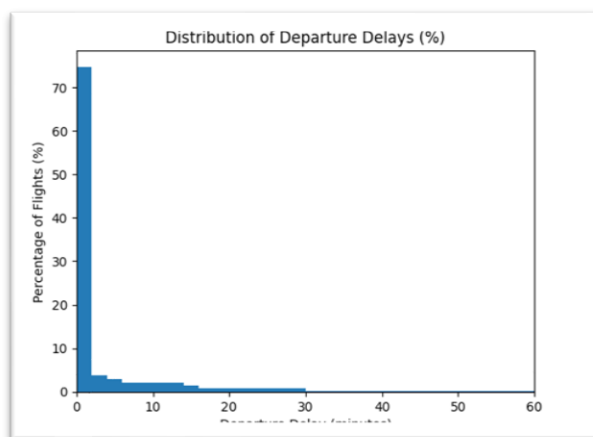
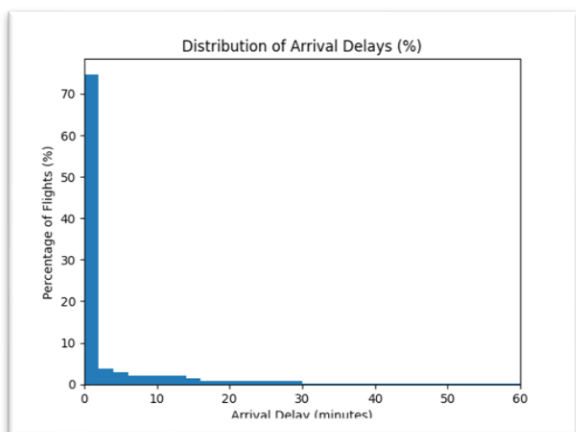
Reg	Campo		Descripción
01	FlightDate		Fecha del vuelo
02	Day_Of_Week		Día de la semana del vuelo
03	Airline		Nombre Aerolínea
04	Tail_Number		ID del vuelo
5-6	Dep_Airport	Arr_Airport	Código Aeropuerto de salida y llegada
7-8	Dep_CityName	Arr_CityName	Ciudad de salida y llegada
09	DepTime_label		Hora original de salida
10-11	Dep_Delay	Arr_Delay	Retraso salida y llegada
12	Dep_Delay_Tag		Marca retraso salida
13-14	Dep_Delay_Type	Arr_Delay_Type	Tipo de retraso salida y llegada
15	Flight_Duration		Duración del vuelo
16	Distance_type		Distancia
17	Delay_Carrier		Retraso por el carrier
18	Delay_Weather		Retraso por condiciones meteorológicas

19	Delay_NAS		Retraso por National Airspace System
20	Delay_Security		Retraso por seguridad
21	Delay_LastAircraft		Retraso por el avión
22	Manufacturer		Fabricante del avión
23	Model		Modelo del avión
24	Aircraft_age		Antigüedad del avión
25-26	Dep_AIRPORT	Arr_AIRPORT	Nombre Aeropuerto de salida y llegada
27-28	Dep_CITY	Arr_CITY	Ciudad de salida y llegada
29-30	Dep_STATE	Arr_STATE	Estado de salida y llegada
31-32	Dep_COUNTRY	Arr_COUNTRY	País de salida y llegada
33-34	Dep_LATITUDE	Arr_LATITUDE	Latitud de salida y llegada
35-36	Dep_LONGITUDE	Arr_LONGITUDE	Longitud de salida y llegada
37-38	Dep_tavg	Arr_tavg	Temperatura promedio en salida y llegada
39-40	Dep_tmin	Arr_tmin	Temperatura mínima en salida y llegada
41-42	Dep_tmax	Arr_tmax	Temperatura máxima en salida y llegada
43-44	Dep_prcp	Arr_prcp	Precipitación en salida y llegada
45-46	Dep_snow	Arr_snow	Cantidad de nieve en salida y llegada
47-48	Dep_wdir	Arr_wdir	Dirección del viento en salida y llegada
29-50	Dep_wspd	Arr_wspd	Velocidad del viento en salida y llegada
51-52	Dep_pres	Arr_pres	Presión barométrica en salida y llegada

4.2.1. Análisis exploratorio y correlaciones (EDA)

4.2.1.1. Exploración de Variables

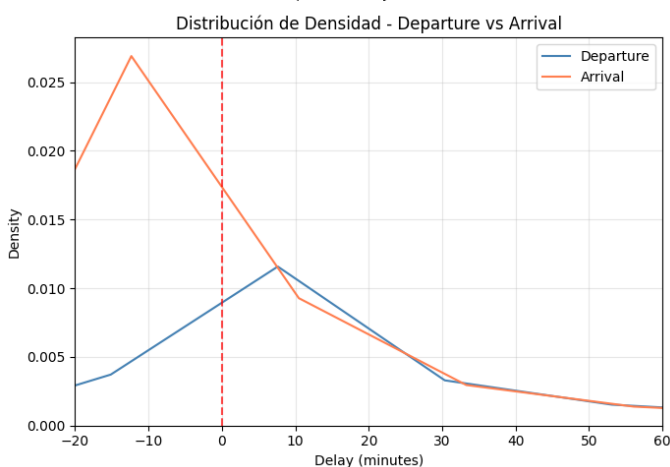
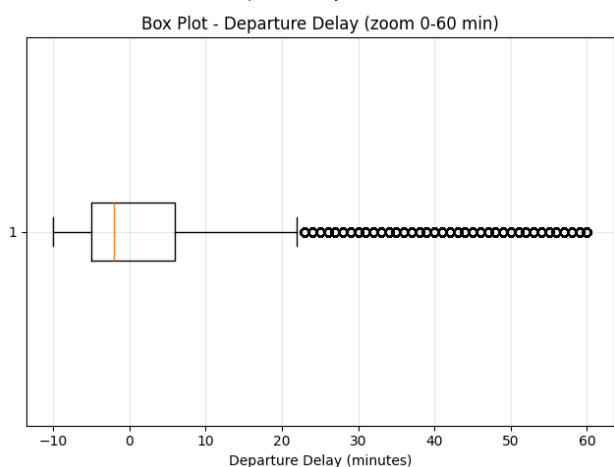
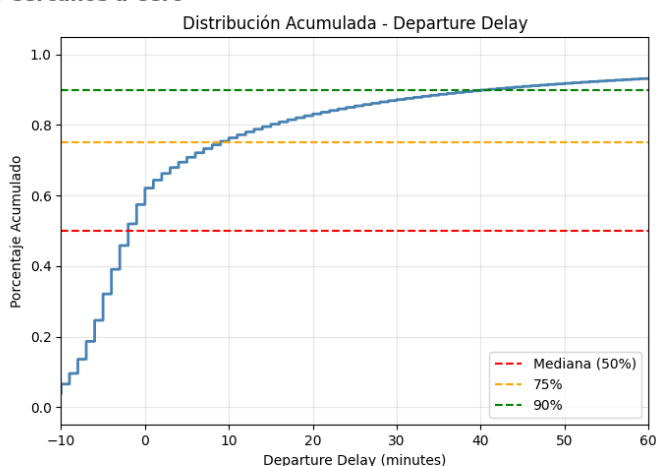
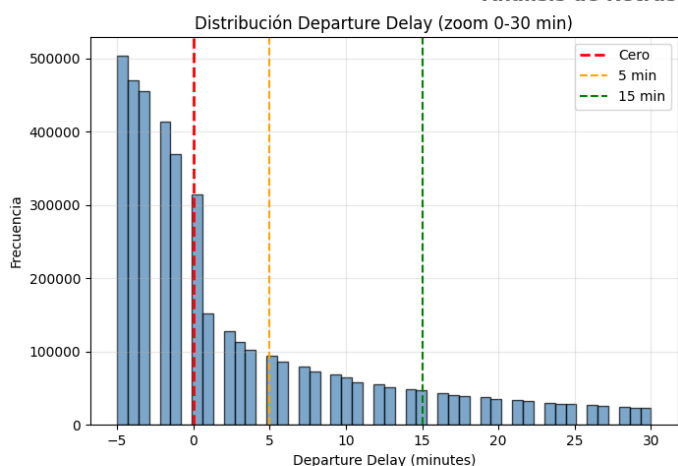
El análisis de la distribución de los retrasos evidencia una alta concentración de vuelos con niveles bajos de demora. En el caso del retraso en salida (dep_delay_min), el 61,3 % de los vuelos presentan retraso igual o inferior a 0 minutos, mientras que el 78,2 % se sitúa por debajo de los 15 minutos. De forma similar, para el retraso en llegada (arr_delay_min), el 61,2 % de los vuelos llegan en horario o antes, y el 77,7 % registra demoras menores o iguales a 15 minutos.

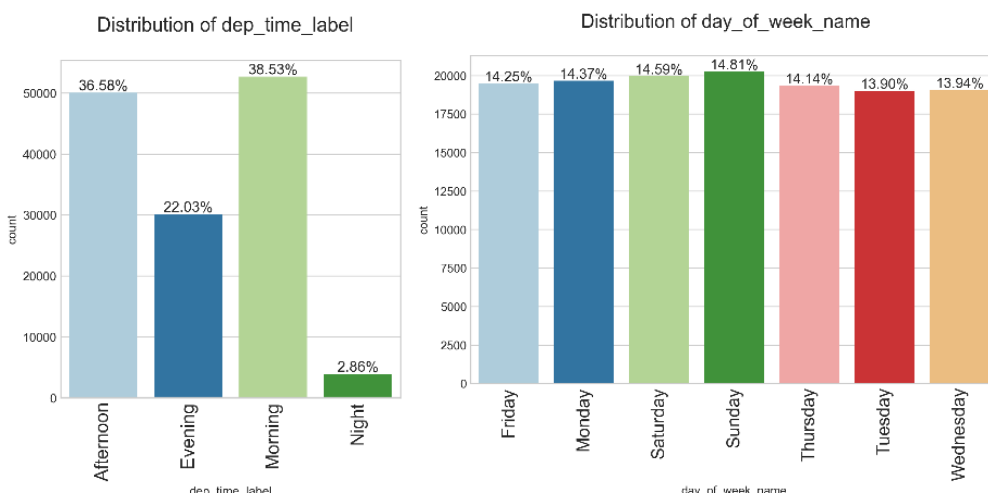


Asimismo, la similitud entre ambas distribuciones sugiere una fuerte relación entre el retraso de salida y el de llegada, coherente con la lógica operacional del sector aéreo.

En términos operacionales, el hecho de que aproximadamente el 22 % de los vuelos superen los 15 minutos de retraso refuerza la relevancia estratégica del sistema, ya que la anticipación temprana de este segmento es donde se concentra el mayor impacto económico y organizativo para las aerolíneas.

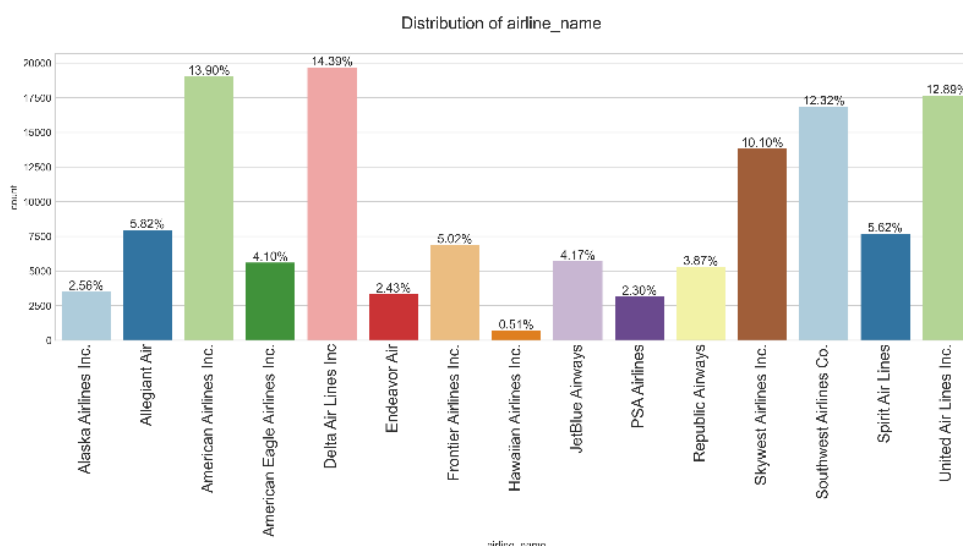
Análisis de Retrasos Cercanos a Cero





Al analizar la distribución temporal de los vuelos según la franja horaria de salida. Se observa que la mayoría de los vuelos están programados durante la mañana (38.53%) y la tarde (36.58%), concentrando casi el 75% del total. Las salidas en la noche representan únicamente un 2.86% del total, lo que indica una baja actividad operativa en esas horas, probablemente por restricciones o menor demanda. Esta concentración en las horas centrales del día refleja la optimización del uso de la infraestructura aeroportuaria y la demanda de pasajeros.

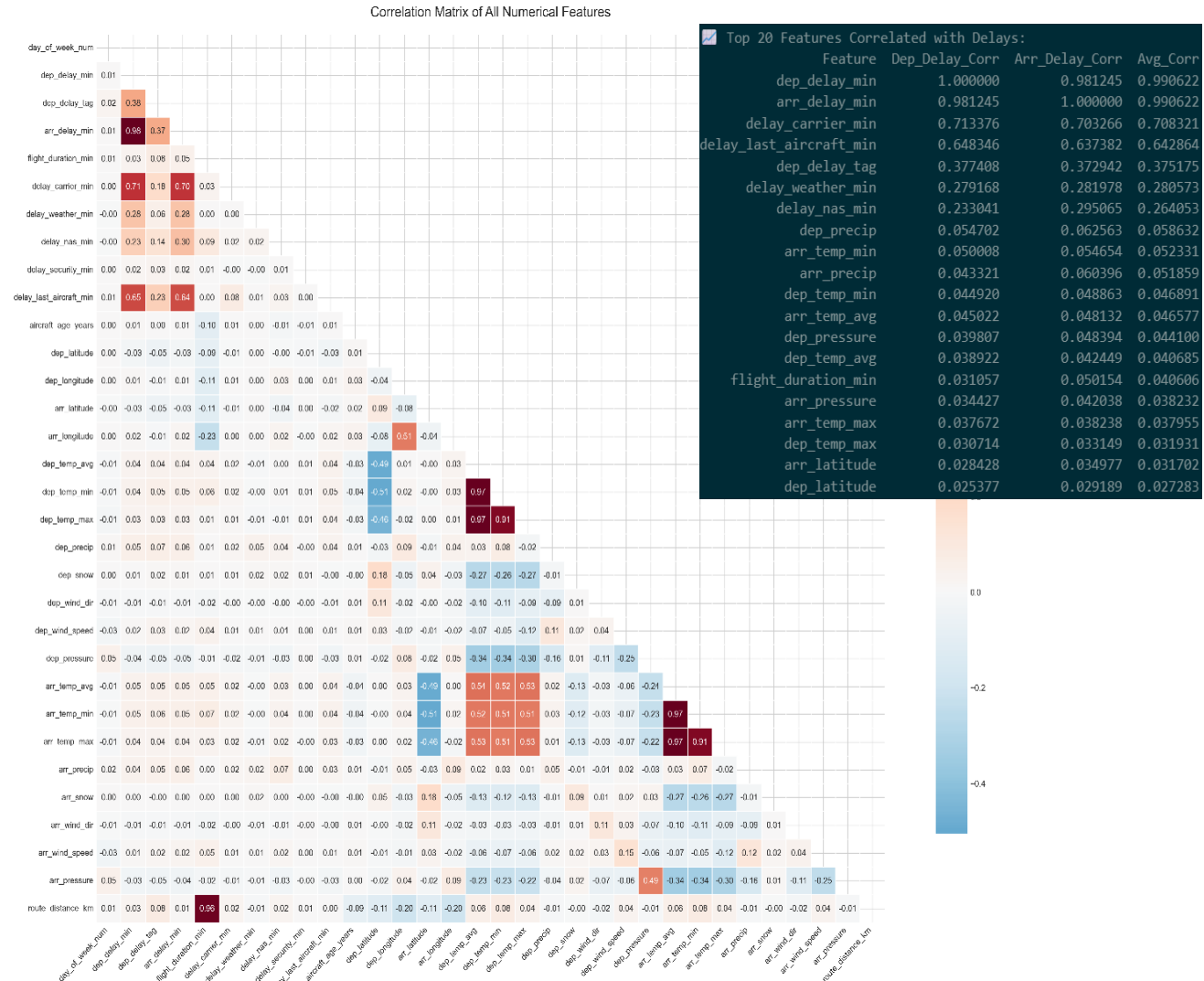
Respecto a la distribución por días de la semana esta resulta ser relativamente homogénea, con porcentajes muy similares que varían entre el 13.90% y el 14.81%. Esto sugiere que la operación aérea mantiene un ritmo constante a lo largo de toda la semana, sin un día con una concentración significativa de vuelos.



Finalmente, la distribución por aerolíneas muestra una mayor heterogeneidad, donde cinco aerolíneas concentran más del 60% de los vuelos del conjunto de datos. Destacan Delta Air Lines Inc. (14.39%), Allegiant Air (13.90%), United Air Lines Inc. (12.89%), Southwest Airlines Co. (12.32%) y Skywest Airlines Inc. (10.10%). Este hecho evidencia que la muestra está dominada por un número reducido de operadores principales, lo cual puede influir en el análisis de retrasos y desempeño.

4.2.2. Correlación de Variables

A continuación, se presenta la matriz de correlación y una tabla que resume las relaciones entre las 20 variables más relevantes del conjunto de datos



A partir de la matriz de correlación de las variables numéricas, se pueden extraer las siguientes conclusiones clave para el análisis del arrival delay (arr_delay_min):

En primer lugar, se observa una correlación muy alta entre el retraso de salida (dep_delay_min) y el retraso de llegada (arr_delay_min), cercana a 1. Esto confirma que los retrasos se propagan a lo largo del vuelo y que el retraso en la salida es el factor más determinante para explicar el retraso final en la llegada.

Asimismo, las variables asociadas a causas operativas del retraso (carrier delay, late aircraft delay, NAS delay y weather delay) presentan correlaciones positivas moderadas con el retraso de llegada. Esto indica que, si bien ninguna de estas variables por sí sola explica completamente el retraso, en conjunto aportan información relevante sobre sus causas estructurales.

Por otro lado, las variables geográficas (latitud y longitud de origen y destino) muestran correlaciones cercanas a cero con el retraso, evidenciando que la ubicación en sí misma no es un factor explicativo directo del retraso, aunque puede ser relevante de forma indirecta cuando se combina con otros factores (clima, congestión aeroportuaria, etc.).

La distancia de la ruta (route_distance_km) presenta una correlación muy baja con el retraso de llegada, lo que indica que vuelos más largos no necesariamente implican mayores retrasos, reforzando la idea de que el retraso está más asociado a factores operativos que a la duración del trayecto. De manera similar, las variables categóricas transformadas en numéricas, como los días de la semana, presentan correlaciones débiles con el retraso de llegada. Esto sugiere que el día del vuelo tiene un impacto limitado y que su efecto probablemente sea no lineal o dependiente de otros factores operativos.

A continuación, se presenta la matriz de correlación de las variables climáticas, la cual permite analizar las relaciones entre los distintos factores meteorológicos registrados.



Las variables relacionadas con la meteorología, especialmente las temperaturas mínimas, medias y máximas tanto en origen como en destino, muestran altas correlaciones entre sí, lo cual era esperable, pero

una relación débil con el retraso de llegada. Esto sugiere que el clima, en términos generales, no impacta directamente en el retraso salvo en eventos específicos (por ejemplo, nieve o viento), que presentan correlaciones bajas, pero no nulas. Dado que las variables de temperatura muestran una fuerte correlación entre sí, se optó por conservar únicamente la temperatura promedio, al considerarse una medida más representativa del comportamiento general del día. Esta decisión permite reducir redundancia en el conjunto de variables, simplificar el modelo sin pérdida significativa de información relevante.

En síntesis, la matriz de correlación confirma que el retraso de llegada está fuertemente influenciado por el retraso de salida y por factores operativos, mientras que variables meteorológicas, geográficas y temporales aportan información complementaria, pero con menor poder explicativo individual. Estos resultados justifican el uso de modelos no lineales, capaces de capturar interacciones complejas entre variables, como los modelos de boosting y KNN que mostraron mejor desempeño predictivo.

4.3. Feature Engineering

A partir del Análisis Exploratorio de Datos, se identificaron variables con capacidad explicativa significativa sobre los retrasos de salida (Dep_Delay) y llegada (Arr_Delay). Se priorizaron variables operativas, temporales y meteorológicas disponibles antes del despegue, asegurando su viabilidad en un escenario real de predicción anticipada.

En base al análisis de correlación presentado en el apartado anterior, se eliminaron las variables de temperatura mínima y máxima, conservándose únicamente la temperatura promedio con el objetivo de reducir redundancia en el conjunto de variables y simplificar el modelo.

Además, el dataset utilizado no presentó valores nulos ni registros duplicados, por lo que no fue necesario aplicar procesos adicionales de limpieza estructural.

4.3.1. Transformación de las variables categóricas

Las variables categóricas fueron transformadas mediante label encoding, el cual le asigna a cada una un valor numérico, permitiendo que los modelos de Machine Learning procesen estas variables. Se optó por este método debido a la alta cardinalidad de algunas variables (por ejemplo, aeropuertos), lo cual haría inviable el uso de One-Hot Encoding por el incremento excesivo de dimensionalidad.

El resumen del encoding de las 6 variables categóricas es:

- Label Encoders airline_name..... 15 unique values
- Label Encoders dep_airport_name..... 349 unique values
- Label Encoders arr_airport_name..... 349 unique values
- Label Encoders day_of_week_name..... 7 unique values
- Label Encoders aircraft_model..... 21 unique values
- Label Encoders aircraft_manufacturer... 5 unique values

La asignación de valores numéricos a variables categóricas es adecuada para modelos basados en árboles, como LightGBM, ya que estos algoritmos realizan divisiones mediante reglas de decisión y no asumen relaciones lineales entre los valores. Sin embargo, esta estrategia puede afectar el desempeño de modelos lineales (Regresión Lineal, Ridge, Lasso) y modelos basados en distancia como KNN. La codificación numérica introduce implícitamente un orden matemático artificial entre categorías nominales (por ejemplo, "Aerolínea 1" < "Aerolínea 2"), cuando en realidad no existe una relación entre ellas. Esto puede generar sesgos en la estimación de coeficientes o distorsiones en el cálculo de distancias, afectando potencialmente la capacidad predictiva de dichos algoritmos.

Como alternativa, se evaluó la aplicación de One-Hot Encoding. No obstante, esta técnica implicaría una expansión significativa del espacio dimensional, ya que variables como aerolínea y aeropuerto contienen cientos de valores únicos. A esto se sumarían fabricantes y modelos de aeronaves, que, si bien presentan menor cardinalidad, contribuirían igualmente al crecimiento exponencial del número de columnas. Esta elevada dimensionalidad aumentaría considerablemente el costo computacional, tanto en memoria como en tiempo de entrenamiento. Por ende, se optó por mantener una codificación numérica controlada, priorizando un equilibrio entre eficiencia computacional y capacidad predictiva

En consecuencia, la técnica de codificación empleada constituye una decisión metodológica que puede influir en el comportamiento diferencial de los modelos evaluados, aspecto que será analizado en la sección de comparación de modelos.

4.3.2. Variables derivadas

Con el fin de capturar patrones temporales, operativos y contextuales que no se encuentran representados en las variables lineales, se generaron variables temporales.

Se generaron las siguientes variables temporales:

- **is_weekend**: indicador binario de fin de semana.

Además, se generaron variables operacionales con el fin de capturar patrones estructurales asociados al funcionamiento propio del sistema aéreo y que no pueden ser capturados por una única variable.

- **route_distance_km**: combinación del aeropuerto de origen y destino.
- **carrier_avg_delay**: retraso promedio histórico por aerolínea.
- **route_avg_delay**: retraso promedio histórico por ruta.
- **dep_delay_lag**: retraso real del vuelo anterior
- **arr_delay_lag**: retraso real del vuelo anterior
- **total_delay_history**: acumulado de retrasos históricos asociados al vuelo

4.3.3. Variables definidas

Tras el proceso de selección, transformación y generación de variables, se definió el conjunto final de features utilizadas para el entrenamiento de los modelos.

- Se identificaron 6 variables categóricas principales, transformadas mediante Label Encoding.
- Departure Delay, se utilizaron 21 variables explicativas, seleccionadas en función de su disponibilidad previa al despegue y su relevancia identificada en el análisis exploratorio.
- Arrival Delay, se emplearon 22 variables explicativas, incorporando información adicional asociada a factores acumulativos del trayecto.

La diferenciación de features entre ambos modelos responde a la naturaleza distinta del fenómeno a predecir, dado que el retraso de llegada incorpora efectos adicionales derivados de la operación completa del vuelo.

Debe señalarse que la inclusión del retraso real de salida como variable explicativa para la predicción del retraso de llegada implica que el modelo se aplica una vez que el vuelo ha despegado y dicha información ya se encuentra disponible. En este contexto, la predicción se orienta a estimar el desvío final durante el desarrollo del vuelo.

Durante la fase de entrenamiento, la variable `dep_delay_lag` fue construida utilizando el retraso real del vuelo inmediatamente anterior, permitiendo modelar el efecto de arrastre operacional entre vuelos consecutivos de una misma aeronave.

Asimismo, las variables asociadas a causas de retraso (`carrier`, `weather`, `NAS`, `security` y `last aircraft`) fueron incorporadas en función de los registros históricos disponibles en el dataset.

Sin embargo, en la fase de predicción dichas variables no siempre se encuentran disponibles en tiempo real para el vuelo a estimar. En consecuencia, al ejecutar el modelo sobre nuevos registros, estos valores se aproximan mediante promedios históricos calculados a partir de combinaciones operativas equivalentes (por ejemplo, ruta y día de la semana), o mediante promedios globales cuando no existen suficientes observaciones específicas.

De este modo, el modelo conserva la capacidad de capturar patrones estructurales asociados tanto al arrastre operacional como a las distintas causas de demora, sin incurrir en la utilización de información futura. Este enfoque permite preservar la validez metodológica del modelo, manteniendo su capacidad predictiva anticipada.

A continuación, se presenta el fragmento de código donde se definen las variables finales utilizadas en el modelo.

```
categorical_cols = ['airline_name',  
                    'dep_airport_name',  
                    'arr_airport_name',  
                    'day_of_week_name',  
                    'aircraft_model',  
                    'aircraft_manufacturer']
```

```
features_for_dep_delay = ['airline_name',  
                           'dep_airport_name',  
                           'arr_airport_name',  
                           'day_of_week_name',  
                           'aircraft_model',  
                           'aircraft_manufacturer',  
                           'aircraft_age_years',  
                           'delay_carrier_min',  
                           'delay_last_aircraft_min',  
                           'dep_delay_lag',  
                           'delay_weather_min',  
                           'dep_temp_avg',    # Average temperature  
                           'dep_precip',      # Precipitation amount  
                           'dep_snow',        # Snow amount  
                           'dep_wind_speed',  # Wind speed  
                           'dep_pressure',    # Barometric pressure  
                           'delay_nas_min',  
                           'route_distance_km',  
                           'flight_duration_min',  
                           'is_weekend',
```

```
'total_delay_history']
```

```
features_for_arr_delay = ['airline_name',  
                           'dep_airport_name',  
                           'arr_airport_name',  
                           'day_of_week_name',  
                           'aircraft_model',  
                           'aircraft_manufacturer',  
                           'aircraft_age_years',  
                           'dep_delay_min',  
                           'delay_carrier_min',  
                           'delay_last_aircraft_min',  
                           'arr_delay_lag',  
                           'delay_weather_min',  
                           'arr_temp_avg',  
                           'arr_precip',  
                           'arr_snow',  
                           'arr_wind_speed',  
                           'arr_pressure',  
                           'delay_nas_min',  
                           'route_distance_km',  
                           'flight_duration_min',  
                           'is_weekend',  
                           'total_delay_history']
```

4.4. Modelos de Machine learning

Para modelar el problema planteado se analizaron los siguientes modelos de Machine Learning:

4.4.1. Linear Regression

Modelo lineal que estima la variable objetivo como combinación lineal de las variables explicativas. Sirve como modelo base por su simplicidad e interpretabilidad.

4.4.2. Ridge

Extensión de la regresión lineal que incorpora regularización L2, añadiendo al término de error una penalización proporcional a la suma de los cuadrados de los coeficientes. Esta penalización reduce la magnitud de los parámetros sin llevarlos exactamente a cero, lo que disminuye el riesgo del overfitting y mejora la estabilidad del modelo ante multicolinealidad.

El grado de penalización se controla mediante el hiperparámetro α , que regula el equilibrio entre ajuste y regularización.

4.4.3. Lasso

Similar a Ridge, pero con regularización L1, lo que permite además realizar selección automática de variables. A diferencia de L2, la penalización L1 puede llevar algunos coeficientes exactamente a cero, permitiendo realizar selección automática de variables. Esto resulta especialmente útil en escenarios con alta dimensionalidad o presencia de variables irrelevantes.

4.4.4. ElasticNet

Combina regularización L1 y L2, integrando la selección automática de variables del Lasso con la estabilidad del Ridge. Permite equilibrar simplicidad y robustez del modelo, siendo especialmente adecuado cuando existen variables altamente correlacionadas, ya que evita la selección arbitraria de una sola de ellas y puede distribuir el peso entre variables relacionadas.

4.4.5. Decision Tree

Modelo no lineal basado en reglas de decisión que divide los datos en subconjuntos según variables explicativas.

4.4.6. Random forest

Ensamble de múltiples árboles de decisión contruidos sobre muestras aleatorias, lo que reduce varianza y mejora robustez.

4.4.7. Extra Trees

Similar a Random Forest, pero introduce mayor aleatoriedad en la selección de splits, reduciendo aún más el overfitting.

4.4.8. AdaBoost

Modelo de boosting que entrena modelos secuenciales corrigiendo los errores de los anteriores.

4.4.9. Gradient Boosting

Algoritmo de boosting que optimiza iterativamente los errores mediante descenso por gradiente.

4.4.10. XGBoost

Implementación optimizada de Gradient Boosting con regularización y alto rendimiento computacional.

4.4.11. LightGBM

Algoritmo de boosting eficiente para grandes volúmenes de datos, basado en crecimiento por hojas.

4.4.12. CatBoost

Modelo de boosting diseñado para manejar variables categóricas de forma eficiente y reducir el overfitting.

4.4.13. KNN

Modelo supervisado que predice el valor de una observación a partir de los k vecinos más cercanos en el espacio de características.

4.5. Estrategia de Entrenamiento

Para evaluar el desempeño de los modelos de Machine Learning, se definieron cuatro posibles configuraciones que combinan distintos tamaños de dataset y cantidad de modelos a entrenar: Hay dos tamaños del dataset, uno tiene aproximadamente 136K registros y otro 6.7M de registros.

Las posibles opciones son:

- (ff) dataset de 6.7M de registros, con todos los modelos (13)
- (fr) dataset de 6.7M de registros, con menos cantidad de modelos (<13)
- (rf) dataset de 136K de registros, con todos los modelos (13)
- (rr) dataset de 136K de registros, con menos cantidad de modelos (<13)

ref.: ff – full full | fr – full reducido | rf – reducido full | rr – reducido reducido

Esta estrategia permitió realizar iteraciones rápidas durante la fase de exploración y ajuste de hiperparámetros, evitando el costo computacional de entrenar repetidamente sobre el dataset completo.

Flight Delay Prediction - Mode Selection

0. EDA Graphs
☐ With EDA graphs ☒ Without EDA graphs ☐ EDA graphs only

1. Select Mode
☐ Train New Models
☒ Use Pre-trained Models (Skip Training)

2. Training Options
☐ Use Every Model (13 models)
☒ Use Full DataFrame (~6.7M rows)

3. Weight Method Selection
Method: none
• Standard training
• All flights weighted equally
• Best for general purpose

4. Additional Info
Info/Notes:
Current config: ~136K rows - <13 models

5. Select Pre-trained Model Set
☐ Full df - 13 Models (~6.7M rows trained)
☐ Full df - < 13 Models (~6.7M rows trained)
☐ Reduced df - 13 Models (~136K rows trained)
☒ Reduced df - < 13 Models (~136K rows trained)
☐ Custom Path

Model Override (optional)
Leave as default to use the best model from each folder.
Select a specific model to override.
Departure model: elasticnet_3.pkl
Arrival model: Best model (default)
Found 9 dep model(s) and 9 arr model(s) in folder

OK Reset Cancel

Una vez identificadas las configuraciones óptimas, el entrenamiento final se realizó utilizando la totalidad de los datos disponibles.

En el conjunto de datos se observa que la mayoría de los vuelos presentan retrasos cercanos a cero, mientras que los retrasos mayores, como los de tres horas, son relativamente pocos. Desde el punto de vista operativo, resulta más relevante predecir con precisión los retrasos significativos, aunque ello implique tolerar pequeñas imprecisiones en los retrasos menores. Dado que la variable objetivo presenta una alta concentración de valores iguales a cero, existe un fuerte desbalance que puede inducir al modelo a optimizar su predicción hacia valores cercanos a cero, reduciendo el error promedio, pero perdiendo sensibilidad ante retrasos relevantes.

Para mitigar este efecto, se incorporó una función de penalización durante el entrenamiento, que asigna mayor peso a los errores en vuelos con retrasos efectivos. De esta manera, el modelo prioriza aprender los casos críticos y mejora la capacidad predictiva en situaciones de mayor impacto operativo, aumentando la utilidad práctica de las predicciones sin comprometer significativamente la precisión en vuelos con retrasos menores.

A continuación, se detallan los distintos modelos de penalización utilizados, así como los parámetros específicos definidos en cada caso.

4.5.1. Penalización lineal

En este esquema, la ponderación asignada a cada observación aumenta proporcionalmente al retraso observado. De esta forma, el peso asociado a la función de pérdida crece de manera lineal con la magnitud del retraso real.

Este enfoque permite incrementar gradualmente la importancia de los vuelos con mayor demora, manteniendo una estructura de crecimiento controlada y estable.

```
def create_progressive_weights(y_train,
                               base_weight=1.0,
                               penalty_start_min=60, # Start penalizing at 60 min
                               max_weight=15.0,      # Cap the maximum penalty
                               scale_factor=0.05):    # Controls how fast penalty grows
    """
    Progressive penalty that scales with delay magnitude.

    Penalty curve:
    - delay <= 60 min → weight = 1.0 (no penalty)
    - delay = 120 min → weight ≈ 4.0 (moderate penalty)
    - delay = 180 min → weight ≈ 7.0 (strong penalty)
    - delay = 300 min → weight ≈ 13.0 (capped at max_weight)
    """
```

4.5.2. Penalización exponencial

En la penalización exponencial, el peso asignado a cada observación crece de forma exponencial con la magnitud del retraso. En consecuencia, pequeños incrementos en retrasos elevados producen aumentos significativos en la penalización asociada.

Este esquema intensifica de manera marcada la priorización de los casos críticos, aumentando la sensibilidad del modelo ante demoras severas. No obstante, si los parámetros no se calibran adecuadamente, puede introducir inestabilidad en el proceso de entrenamiento.

```
def create_exponential_weights(y_train,
                               base_weight=1.0,
                               penalty_start_min=60,
                               max_weight=20.0,
                               exp_base=1.015):      # Controls steepness
    """
    Exponential growth in penalty – more aggressive for extreme delays.

    Penalty curve:
    - delay <= 60 min → weight = 1.0
    - delay = 120 min → weight ≈ 2.5
    - delay = 180 min → weight ≈ 6.0
    - delay = 300 min → weight ≈ 36.0 (capped at max_weight)
    """
```

4.5.3. Penalización por niveles

La penalización por niveles consiste en segmentar la variable objetivo en distintos tramos y asignar a cada uno de ellos un peso fijo específico dentro de la función de pérdida. A diferencia de las penalizaciones continuas (lineal o exponencial), este enfoque introduce incrementos discretos en la ponderación.

Este mecanismo permite controlar explícitamente la importancia relativa de diferentes rangos de retraso, alineando el entrenamiento del modelo con criterios operativos previamente definidos. Además, aporta mayor estabilidad frente a valores atípicos, al evitar crecimientos desproporcionados en la penalización.

```
def create_tiered_weights(y_train):  
    """  
    Operationally meaningful tiers aligned with airline delay categories.  
  
    Tier structure mirrors real-world impact:  
    - Early / On-time ( <= 15 min) → 1.0 (normal)  
    - Minor delay ( 15- 60 min) → 2.0 (moderate attention)  
    - Significant ( 60-120 min) → 4.0 (connection risk)  
    - Severe (120-180 min) → 7.0 (high disruption)  
    - Very severe (180-300 min) → 12.0 (major disruption)  
    - Extreme ( >300 min) → 20.0 (crisis level)  
  
    Within each tier, weight scales LINEARLY – no hard jumps.  
    """
```

4.5.4. Penalización raíz cuadrada

En este esquema, la ponderación crece de forma sublineal respecto al retraso observado, generalmente mediante una función raíz cuadrada. Esto implica que el peso aumenta a medida que lo hace el retraso, pero a una tasa decreciente.

Este enfoque introduce una corrección moderada al desbalance de la variable objetivo, incrementando la relevancia de los retrasos positivos sin amplificar excesivamente la influencia de valores elevados.

```
def create_sqrt_weights(y_train,  
                        base_weight=1.0,  
                        penalty_start_min=30,  
                        scale_factor=0.15,  
                        max_weight=15.0):  
    """  
    Square root scaling – fast growth at first, then tapers off.  
  
    Good when you want to penalize medium delays heavily  
    without letting extreme outliers dominate training.  
  
    Penalty curve:  
    - delay <= 30 min → weight = 1.0  
    - delay = 60 min → weight ≈ 1.8  
    - delay = 120 min → weight ≈ 2.5  
    - delay = 300 min → weight ≈ 3.9  
    """
```

4.6. Hiperparámetros Definidos

Se seleccionaron los hiperparámetros probando distintos valores en conjuntos de validación, buscando minimizar MAE y RMSE sin comprometer demasiado los tiempos de entrenamiento y predicción.

A continuación, se muestra un ejemplo de fine-tuning con LightGBM, donde pequeñas modificaciones en profundidad y número de árboles generaron diferencias significativas en las métricas:

Model	MAE	MSE	RMSE	R ²	Train Time (s)	Pred Time (s)	Predictions/sec
LightGBM 3	5.521564	74.304664	8.620015	0.985975	1.241553	0.092518	2.956819e+05
LightGBM 2	7.597895	117.952090	10.860575	0.977736	0.515695	0.023791	1.149854e+06
LightGBM 1	30.519676	4317.950277	65.711112	0.184959	0.154171	0.008000	3.419544e+06

Modelo	n_estimators	lr	max_depth	num_leaves	subsamples	colsample	lambda	freq
LightGBM 1	100	0.01	1	10	0.1	0.1	0.0	0
LightGBM 2	250	0.03	3	30	0.5	0.5	0.5	2
LightGBM 3	500	0.05	6	63	0.8	0.8	1.0	1

Debajo, se incluye el fragmento de código donde se definieron los hiperparámetros para cada modelo.

```
# 6.7M Rows - Large
model_map_ff = {# Linear
    'Linear Regression': make_pipeline(StandardScaler(),
                                      LinearRegression()),

    'Ridge': make_pipeline(StandardScaler(),
                          Ridge(alpha=1.0)),
    #random_state=random_state)),

    'Lasso': make_pipeline(StandardScaler(),
                          Lasso(alpha = 0.1,
                              max_iter = 1000)),
    #random_state = random_state)),

    'ElasticNet': make_pipeline(StandardScaler(),
                              ElasticNet(alpha = 0.5,
                                      l1_ratio = 0.5,
                                      max_iter = 1000,
                                      tol=1e-3)),
    #random_state = random_state)),

    # Tree
    'Decision Tree': DecisionTreeRegressor(max_depth=8,
                                          max_features='sqrt',
                                          random_state=random_state),

    'Random Forest': RandomForestRegressor(n_estimators=100,
                                          max_depth=12,
                                          min_samples_leaf=50,
                                          max_features='sqrt',
                                          random_state=random_state,
                                          n_jobs=-1),
    # with those parameters the size of the pkl file is around 1.3GB

    'Extra Trees': ExtraTreesRegressor(n_estimators=100,
                                       max_depth=12,
                                       min_samples_leaf=50,
                                       max_features='sqrt',
                                       random_state=random_state,
                                       n_jobs=-1),
    # with those parameters the size of the pkl file is around 79GB
    # 400
    # None
    # 2
}
```

```
# Boosting
'AdaBoost': AdaBoostRegressor(n_estimators=50,
                              learning_rate=0.5,
                              random_state=random_state),

'Gradient Boosting': GradientBoostingRegressor(n_estimators=75,
                                                learning_rate=0.2,
                                                max_depth=4,
                                                subsample=0.8,
                                                min_samples_leaf=50,
                                                random_state=random_state),

'LightGBM': LGBMRegressor(n_estimators=300,          # 300 → 1000, let early stopping decide
                          learning_rate=0.1,        # 0.1 → 0.05, slower but better generalization
                          num_leaves=128,           # 64 → 128, more capacity for 6.7M rows
                          subsample=0.8,
                          subsample_freq=1,         # makes subsample actually work
                          colsample_bytree=0.8,
                          reg_lambda=1.0,
                          reg_alpha=0.1,           # L1 regularization alongside L2
                          min_child_samples=50,     # prevents overfitting at 6.7M rows
                          random_state=random_state,
                          n_jobs=-1,
                          verbose=-1),

'XGBoost': XGBRegressor(n_estimators=1000,          # 300 → 1000, match LightGBM budget
                        learning_rate=0.05,         # 0.1 → 0.05, match LightGBM for fair comparison
                        max_depth=6,                # 8 → 6, reduces overfitting and file size
                        subsample=0.8,
                        colsample_bytree=0.8,
                        min_child_weight=50,        # equivalent to LightGBM min_child_samples
                        reg_lambda=1.0,              # L2 regularization
                        reg_alpha=0.1,              # L1 regularization
                        tree_method='hist',
                        random_state=random_state,
                        n_jobs=-1),

'CatBoost': CatBoostRegressor(iterations=1000,
                               learning_rate=0.05,
                               depth=6,
                               l2_leaf_reg=3,
                               min_data_in_leaf=50,
                               bootstrap_type='Bernoulli',
                               subsample=0.8,
                               random_state=random_state,
                               verbose=0),

# Distance
'KNN': make_pipeline(StandardScaler(),
                     KNeighborsRegressor(n_neighbors=7,
                                         weights='distance',
                                         # algorithm='ball_tree',
                                         n_jobs=-1)))
```

4.7. Entrenamiento

Para entrenar el modelo se dividieron los datos de la siguiente manera:

- Entrenamiento: 70% (entrenamiento de modelo)
- Validación:15% (ajuste de hiperparámetros)
- Test:15% (evaluación final)

4.7.1. Métricas de Evaluación

Los modelos se evaluaron con métricas estándar de regresión para cuantificar precisión y eficiencia:

- MAE (Mean Absolute Error): Error absoluto promedio en minutos.
- MSE (Mean Squared Error): Error cuadrático medio.
- RMSE (Root Mean Squared Error): Raíz del error cuadrático medio, más interpretable en minutos.
- R^2 : Porcentaje de varianza explicada por el modelo.
- MAPE (Mean Absolute Percentage Error): Error porcentual absoluto medio.
- Train Time (sec): Tiempo de entrenamiento.
- Pred Time (sec): Tiempo de predicción por conjunto de prueba.

- Predictions/sec: Velocidad de predicción por segundo.

Estas métricas permiten comparar modelos no solo en precisión sino también en eficiencia computacional, lo que es crítico para aplicaciones en tiempo real en aerolíneas.

4.7.2. Pruebas Realizadas

Se obtuvieron los siguientes resultados para la predicción de retrasos en minutos

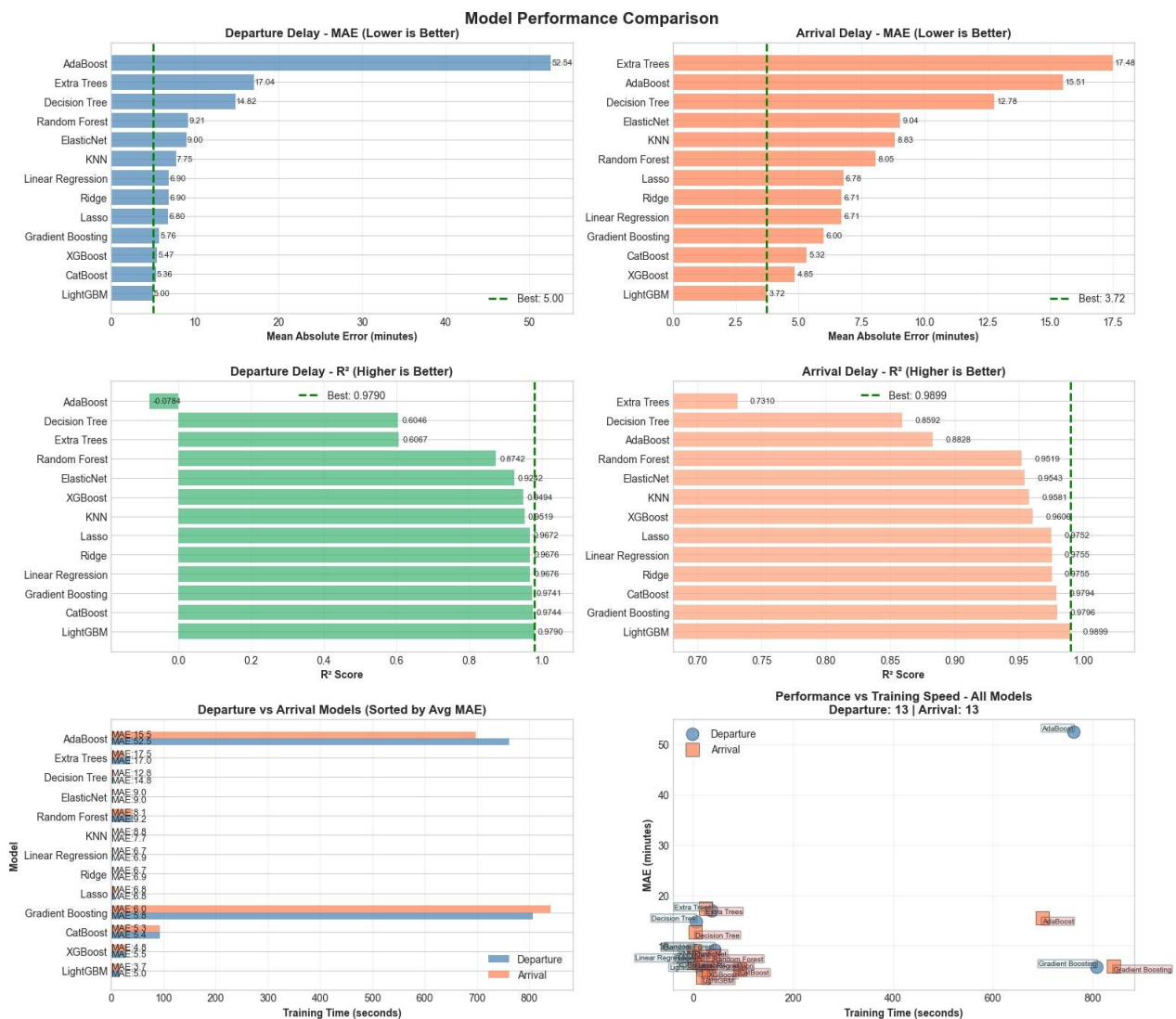
4.7.2.1. Resultados Predicción Retraso de Salida y Llegadas

Los resultados para los diferentes modelos de salida se pueden observar en la siguiente imagen

	Model	Target	MAE	MSE	RMSE	R ²	MAPE	SMAPE	Train Time (s)	Pred Time (s)	Predictions/sec
0	LightGBM	dep_delay_min	5.004356	61.858287	7.865004	0.979010	9999.0	102.128908	16.975365	0.880644	1.531464e+06
1	CatBoost	dep_delay_min	5.363692	75.561068	8.692587	0.974360	9999.0	107.004809	93.358256	0.053998	2.497617e+07
2	XGBoost	dep_delay_min	5.471550	149.137451	12.212185	0.949394	9999.0	105.227430	27.621345	0.673688	2.001927e+06
3	Gradient Boosting	dep_delay_min	5.764022	76.406078	8.741057	0.974074	9999.0	111.330196	807.487533	0.824170	1.636405e+06
4	Lasso	dep_delay_min	6.798882	96.612426	9.829162	0.967217	9999.0	116.587940	6.381788	0.133017	1.013913e+07
5	Ridge	dep_delay_min	6.895564	95.626256	9.778868	0.967552	9999.0	115.708411	1.638047	0.137539	9.805771e+06
6	Linear Regression	dep_delay_min	6.895568	95.626255	9.778868	0.967552	9999.0	115.708415	2.483614	0.142135	9.488681e+06
7	KNN	dep_delay_min	7.746439	141.667713	11.902425	0.951929	9999.0	113.314422	1.474239	1299.762889	1.037632e+03
8	ElasticNet	dep_delay_min	8.998865	223.374714	14.945726	0.924204	9999.0	147.156329	2.138974	0.136616	9.872014e+06
9	Random Forest	dep_delay_min	9.210502	370.683622	19.253146	0.874219	9999.0	142.158319	42.710799	0.370703	3.638155e+06
10	Decision Tree	dep_delay_min	14.819636	1165.309843	34.136635	0.604585	9999.0	157.706785	5.669614	0.118017	1.142783e+07
11	Extra Trees	dep_delay_min	17.040026	1159.115876	34.045791	0.606686	9999.0	152.011656	37.180162	1.851388	7.284669e+05
12	AdaBoost	dep_delay_min	52.539475	3178.201116	56.375537	-0.078434	9999.0	163.876285	761.002066	3.692266	3.652703e+05

Los resultados para los diferentes modelos de llegada se pueden observar en la siguiente imagen

	Model	Target	MAE	MSE	RMSE	R ²	MAPE	SMAPE	Train Time (s)	Pred Time (s)	Predictions/sec
0	LightGBM	arr_delay_min	3.723846	32.057038	5.661894	0.989889	9999.0	47.226833	18.662788	0.960424	1.404250e+06
1	XGBoost	arr_delay_min	4.849016	124.928566	11.177145	0.960596	9999.0	53.779065	30.484816	0.669642	2.014023e+06
2	CatBoost	arr_delay_min	5.317530	65.245574	8.077473	0.979421	9999.0	57.795575	93.292638	0.052188	2.584255e+07
3	Gradient Boosting	arr_delay_min	5.997497	64.681077	8.042455	0.979599	9999.0	62.035468	841.048920	1.091542	1.235569e+06
4	Linear Regression	arr_delay_min	6.707608	77.772927	8.818896	0.975470	9999.0	70.041091	2.729871	0.164318	8.207733e+06
5	Ridge	arr_delay_min	6.707610	77.772927	8.818896	0.975470	9999.0	70.041124	1.817118	0.151257	8.916459e+06
6	Lasso	arr_delay_min	6.781371	78.663994	8.869272	0.975188	9999.0	70.204964	8.747560	0.155706	8.661680e+06
7	Random Forest	arr_delay_min	8.053282	152.343103	12.342735	0.951949	9999.0	73.646816	40.558602	0.344810	3.911357e+06
8	KNN	arr_delay_min	8.831458	132.968230	11.531185	0.958060	9999.0	81.912965	1.556020	879.003529	1.534323e+03
9	ElasticNet	arr_delay_min	9.038501	144.750213	12.031218	0.954344	9999.0	81.945308	2.458717	0.163666	8.240386e+06
10	Decision Tree	arr_delay_min	12.777931	446.268207	21.125061	0.859242	9999.0	93.855442	4.884066	0.101688	1.326282e+07
11	AdaBoost	arr_delay_min	15.513266	371.708098	19.279733	0.882759	9999.0	144.729978	698.226061	3.395122	3.972391e+05
12	Extra Trees	arr_delay_min	17.484075	852.714822	29.201281	0.731044	9999.0	153.215858	24.569475	0.323825	4.164825e+06



Los resultados del análisis comparativo de modelos para la predicción del retraso de llegada en minutos (arr_delay_min) evidencian que el problema presenta una alta capacidad de modelado, con múltiples algoritmos alcanzando valores elevados de R^2 y errores relativamente bajos. Esto sugiere que las variables utilizadas contienen información suficiente para explicar de forma adecuada los retrasos en la llegada de los vuelos. De manera similar, los resultados para la predicción del retraso de salida (dep_delay_min) muestran un comportamiento comparable, indicando que los modelos mantienen un desempeño consistente para ambos objetivos.

Para la predicción del retraso de salida (dep_delay_min), LightGBM alcanza el mejor desempeño global, con un MAE de 5.00 minutos y un R^2 de 0.9790, superando al resto de los modelos tanto en precisión como en estabilidad. De forma consistente, en la predicción del retraso de llegada (arr_delay_min), vuelve a posicionarse como la mejor alternativa, obteniendo un MAE de 3.72 minutos y un R^2 de 0.9899, lo que indica una capacidad explicativa extremadamente elevada.

Los modelos de boosting, como CatBoost, XGBoost y Gradient Boosting, también muestran resultados competitivos, aunque ligeramente inferiores. Esto refuerza la evidencia de que los métodos basados en árboles capturan de forma eficaz las relaciones no lineales presentes en el fenómeno de propagación de

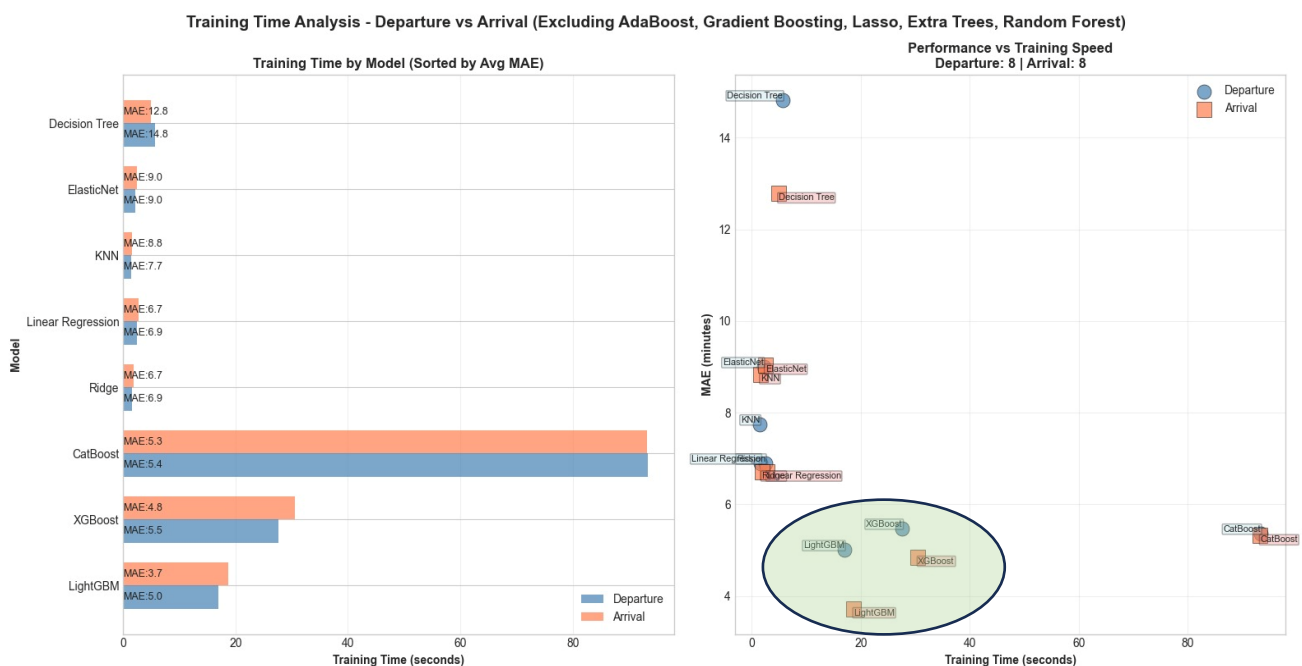
retrasos. LightGBM destaca por su alta capacidad predictiva y por permitir una elevada tasa de predicciones por segundo y su tiempo de entrenamiento, mientras que CatBoost, presenta tiempo de predicción aún menores, siendo particularmente apropiado para entornos de alta demanda operativa.

Por su parte, Gradient Boosting logra buenos resultados en términos de precisión, pero presenta un costo computacional excesivo en el entrenamiento, lo que limita su aplicabilidad práctica. Los modelos basados en árboles, como Random Forest y Decision Tree, muestran un desempeño estable, aunque sin superar a las técnicas de boosting más modernas.

Por su parte, el modelo KNN, que en pruebas preliminares mostró resultados prometedores en función del R^2 , sin embargo, su tiempo de predicción resultó extremadamente alto (1299 segundos), lo que lo hace poco viable para aplicaciones en tiempo real. Esta situación se explica por la propia naturaleza del algoritmo: KNN no construye una función paramétrica ni una estructura de decisión, sino que almacena íntegramente el conjunto de entrenamiento (6,7 millones de registros) y calcula, para cada nueva observación, la distancia respecto de todos los casos almacenados con el fin de identificar los vecinos más cercanos. Este proceso implica un costo computacional elevado en la fase de inferencia, especialmente en datasets de gran tamaño.

Los modelos lineales (Linear Regression, Ridge, Lasso y ElasticNet) presentan resultados consistentes y tiempos de ejecución reducidos, funcionando adecuadamente como modelos base o de referencia. Si bien no alcanzan la precisión de los modelos más complejos, aportan simplicidad, interpretabilidad y menor costo computacional.

Finalmente, AdaBoost presenta un desempeño claramente deficiente, con errores elevados y un R^2 negativo, lo que indica que no logra capturar la dinámica del retraso de llegada y resulta inadecuado para este problema.

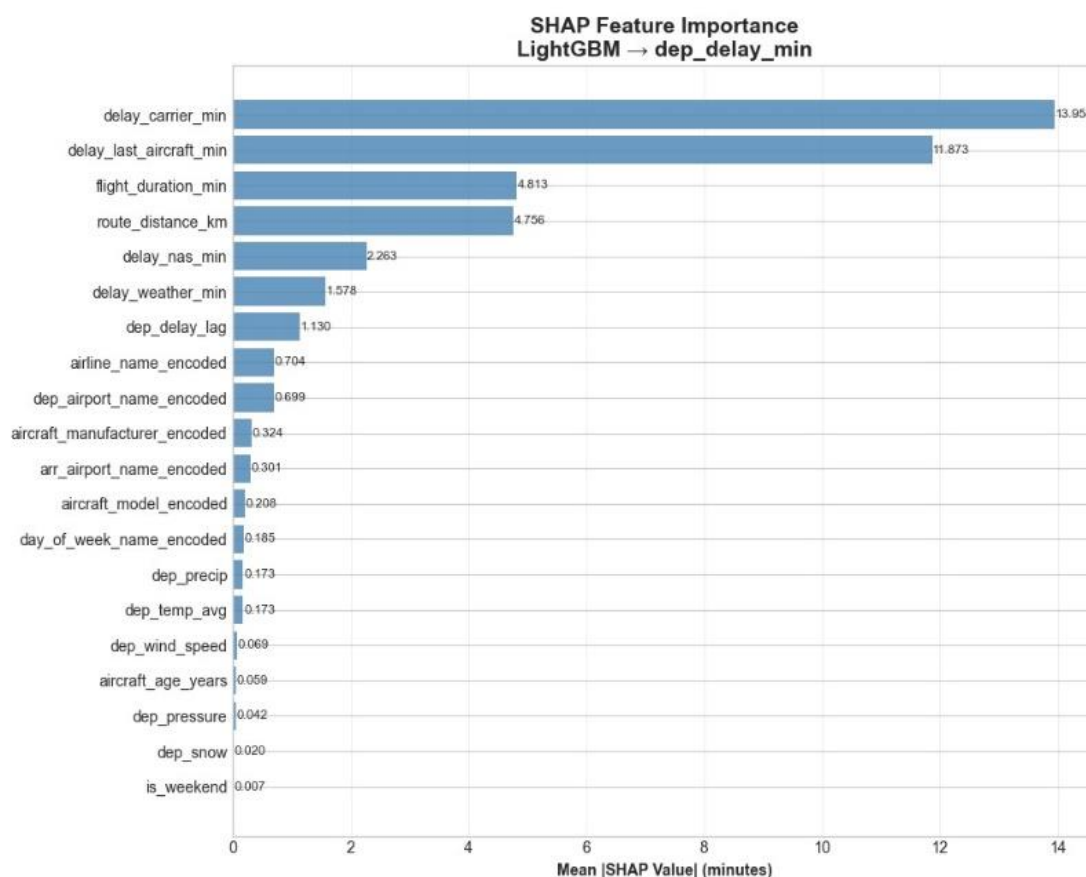


La incorporación de una penalización diferencial en la función de pérdida produjo un leve incremento del MAE global. Sin embargo, se observó una mejora significativa en la capacidad del modelo para estimar retrasos positivos, reduciendo la subestimación sistemática en vuelos con demora efectiva.

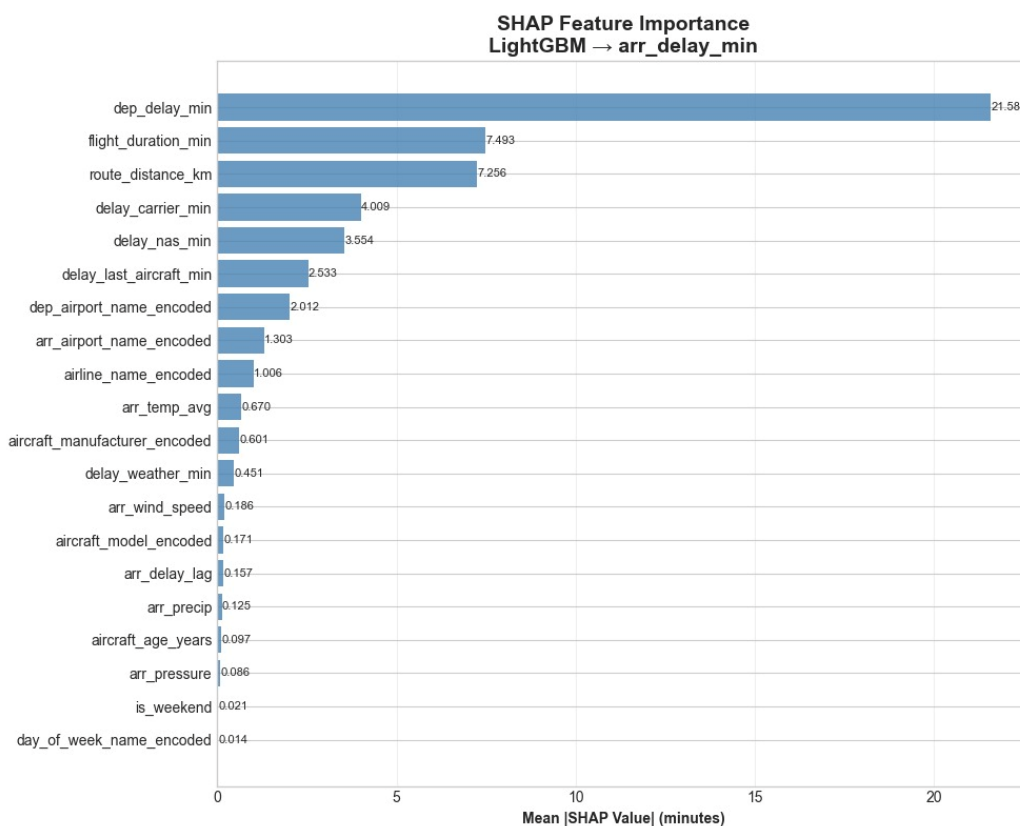
Este comportamiento es consistente con la distribución del dataset, donde una alta proporción de vuelos presenta retraso cero. En ausencia de penalización, el modelo tiende a optimizar el error promedio favoreciendo predicciones cercanas a cero. La estrategia implementada permite equilibrar este sesgo y mejorar la precisión en los casos operativamente más críticos

4.7.2.2. Interpretabilidad del modelo – Análisis SHAP

Con el objetivo de analizar la contribución individual de cada variable en la predicción de retrasos, se aplicó la técnica SHAP. Esta metodología permite cuantificar el impacto promedio de cada variable sobre la predicción final del modelo, proporcionando una interpretación consistente.

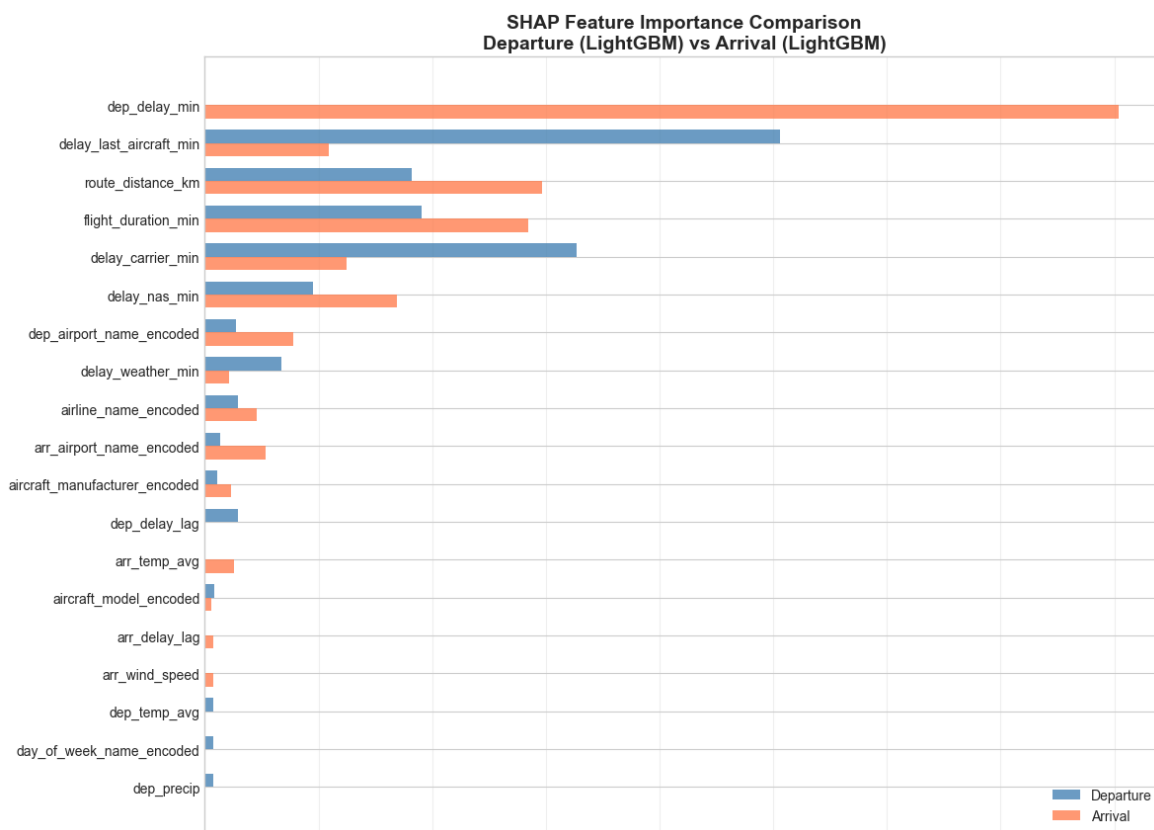


Para el modelo de retraso en salida (dep_delay_min), las variables más influyentes corresponden a los retrasos atribuibles a la aerolínea (delay_carrier_min) y al retraso del vuelo previo de la misma aeronave (delay_last_aircraft_min), confirmando la importancia de los factores operativos internos y la propagación estructural de demoras.



En el caso del modelo de predicción de retraso en llegada (arr_delay_min), se observa que la variable con mayor influencia es el retraso en salida (dep_delay_min), lo cual evidencia el fuerte efecto de propagación de demoras entre fases consecutivas del vuelo. También presentan alta relevancia la duración del vuelo y la distancia de la ruta.

Si bien la variable route_distance_km no presenta una correlación lineal directa elevada con el retraso, sí muestra una alta correlación con la duración del vuelo. Esto sugiere un efecto indirecto, donde la distancia influye sobre el retraso a través de su relación estructural con la duración. Mientras que la matriz de correlación únicamente captura asociaciones lineales directas, el modelo basado en árboles es capaz de identificar interacciones y efectos combinados entre variables, lo que explica su relevancia dentro del análisis SHAP.



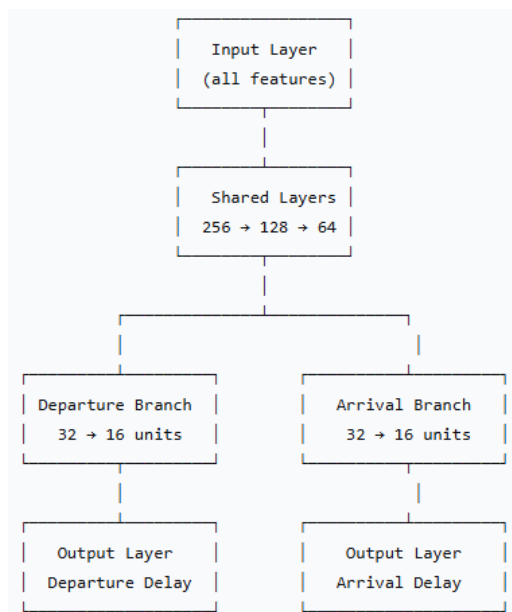
4.8. Pruebas extras con redes neuronales

Con el objetivo de evaluar la capacidad de aprendizaje de patrones complejos y no lineales, se implementó un modelo de Multi-Task Learning con dos salidas: una para predecir el retraso de salida (departure_delay) y otra para el retraso de llegada (arrival_delay).

La incorporación de redes neuronales responde a la necesidad de evaluar si un modelo con mayor capacidad de representación puede capturar interacciones complejas entre variables temporales, operativas y meteorológicas.

4.8.1.Arquitectura del modelo

- Capa de entrada: 47 features numéricas y codificadas a partir de variables categóricas.
- Bloques ocultos: Tres capas densas con normalización por lotes (BatchNormalization) y regularización mediante Dropout, con tamaños 256, 128 y 64 neuronas respectivamente.
- Subcapas específicas: Dos ramas densas adicionales por cada salida, que reducen dimensionalidad hasta las predicciones finales (32 y 16 neuronas).



El modelo fue entrenado igual que los modelos anteriores y al ser evaluado sobre el conjunto de test, se obtuvieron los siguientes resultados.

Métrica	Retraso de salida (Departure Delay)	Retraso de llegada (Arrival Delay)
MAE	6.89 min	5.63 min
RMSE	9.17 min	8.14 min

La red neuronal obtuvo un desempeño competitivo, particularmente en la predicción del retraso de llegada, lo que indica que el modelo es capaz de capturar patrones operativos y temporales complejos. No obstante, su MAE resulta superior al alcanzado por modelos como KNN o LightGBM, que presentan un mejor rendimiento predictivo individual.

La principal ventaja de la red neuronal radica en su enfoque de Multi-Task Learning, ya que permite predecir simultáneamente el retraso de salida y el retraso de llegada mediante un único modelo, aprovechando las relaciones compartidas entre ambas tareas. Sin embargo, tiene un mayor costo computacional, durante la fase de entrenamiento.

En consecuencia, si bien la red neuronal demuestra capacidad para modelar relaciones no lineales complejas, no se considera la alternativa óptima para este problema, ya que LightGBM presenta un mejor desempeño predictivo y mayor eficiencia computacional, logrando menores errores con tiempos de entrenamiento significativamente inferiores.

4.9. Tecnologías y entorno de desarrollo

La solución fue desarrollada utilizando Python versión 3.12.x como lenguaje principal, aprovechando su robustez y amplio ecosistema de librerías para Machine Learning. Para la interacción con el usuario y la exposición de la funcionalidad del sistema se implementó una interfaz web mediante Gradio, lo que permite una visualización sencilla y accesible de los resultados sin necesidad de conocimientos técnicos avanzados.

Todo el código fuente, junto con la estructura del proyecto y la documentación asociada, fue versionado y publicado en un repositorio de GitHub, disponible en <https://github.com/whaleskin/obs-tfm.git>

Se aprovechó la arquitectura modular de Gradio Blocks, junto con handlers de eventos y elementos interactivos, para integrar de manera eficiente los modelos de predicción de retrasos de salida y llegada. El modelo de salida fue entrenado con 21 variables explicativas, mientras que el modelo de llegada utilizó 22 variables, reflejando diferencias específicas en la relevancia de determinadas características según el momento del vuelo.

No obstante, la arquitectura de inferencia fue diseñada para requerir únicamente 10 variables de entrada directa en tiempo real en ambos casos. Las variables restantes corresponden a métricas históricas agregadas (por ejemplo, retrasos promedio por aerolínea o por ruta), las cuales son precomputadas y almacenadas en tablas auxiliares, integrándose automáticamente durante la predicción. Esta separación entre variables dinámicas y estructurales permite mantener la capacidad predictiva del modelo sin comprometer la simplicidad operativa ni la viabilidad de despliegue.

A continuación, se presenta una captura de pantalla de la plataforma desarrollada en Gradio, donde se visualiza la interfaz interactiva utilizada para la generación de predicciones en tiempo real.

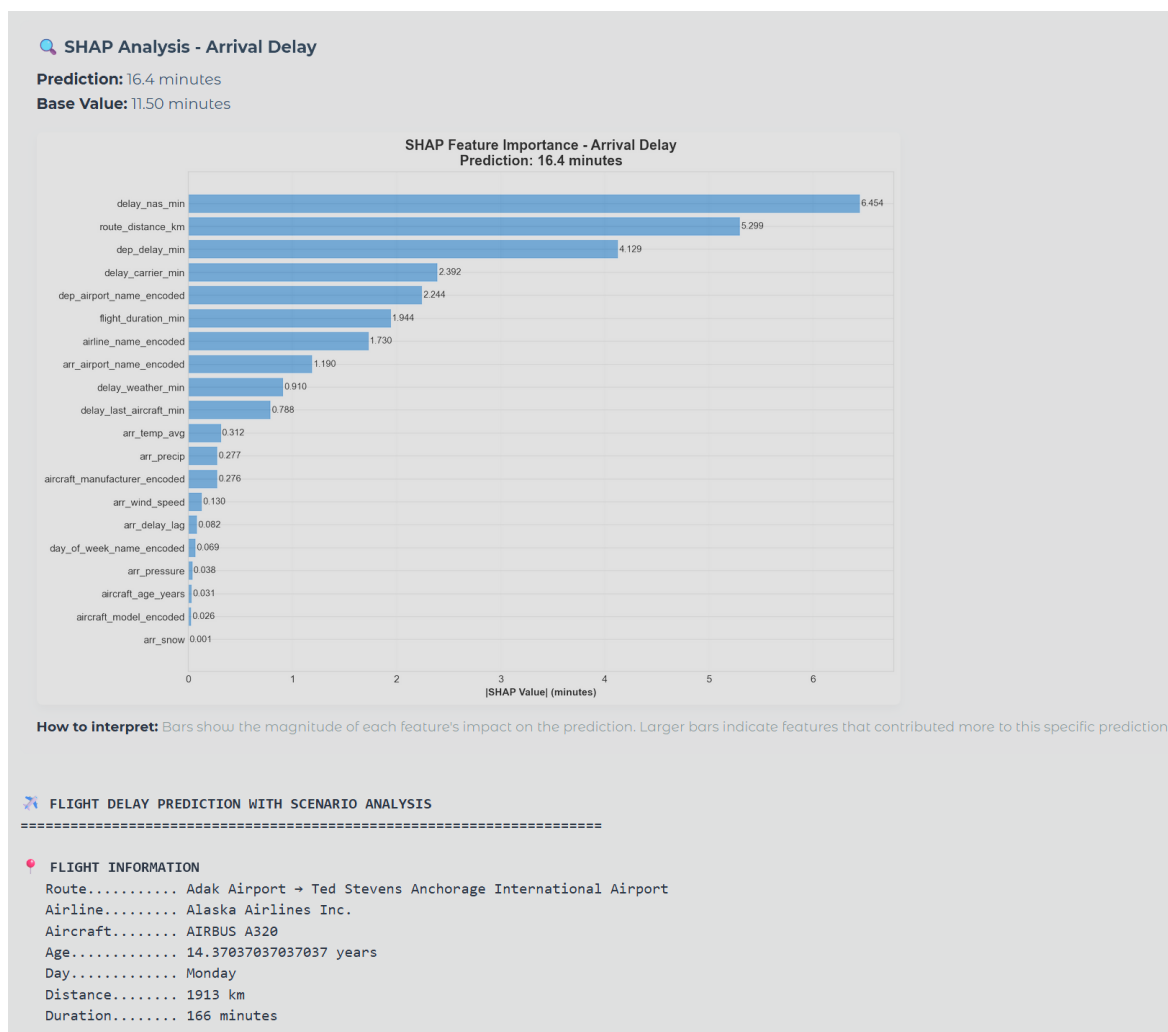
Por otro lado, la interfaz incorpora la posibilidad de simular distintos escenarios climáticos desde el punto de vista visual. Actualmente, esta funcionalidad no impacta en el cálculo del modelo, sino que actúa como una preparación estructural para futuras integraciones con fuentes externas en tiempo real.

The screenshot displays a user interface for weather and scenario analysis. It is divided into three main sections: 'Departure Weather', 'Arrival Weather', and 'Scenario Analysis'.
1. **Departure Weather**: Features a dropdown menu titled 'Weather Condition at Departure' with the instruction 'Select current/forecasted weather at departure airport'. The selected option is 'Clear'.
2. **Arrival Weather**: Features a dropdown menu titled 'Weather Condition at Arrival' with the instruction 'Select current/forecasted weather at arrival airport'. The selected option is 'Clear'.
3. **Scenario Analysis**: Features a dropdown menu titled 'Prediction Scenario' with the instruction 'Select a scenario'. The selected option is 'typical'.
At the bottom, there is a checkbox labeled 'Show SHAP Explanations' with the subtext 'Generate detailed feature importance for this prediction'. The checkbox is currently checked.

Adicionalmente, se prevé la integración futura de fuentes externas en tiempo real, como APIs meteorológicas, lo que permitiría alimentar automáticamente las variables climáticas y reforzar el carácter operativo del sistema en entornos productivos

Por otra parte, la interfaz incorpora funcionalidades de simulación de escenarios (optimista, base y pesimista). La misma se implementa mediante la aplicación de factores de ajuste sobre la predicción base del modelo (por ejemplo, $\pm 25\%$). Esto permite realizar análisis de sensibilidad de forma sencilla, evaluando cómo podrían variar los resultados ante condiciones más favorables o adversas. Si bien estos escenarios no implican un reentrenamiento del modelo, constituyen una herramienta práctica para la planificación operativa y la gestión bajo incertidumbre.

Además, la plataforma integra el análisis SHAP, lo que permite interpretar la contribución individual de cada variable en la predicción generada. Esta capacidad explicativa refuerza la transparencia del sistema y facilita su adopción en entornos empresariales, donde la interpretabilidad resulta un factor clave.



En resumen, la implementación en Gradio transforma el modelo de Machine Learning en una herramienta empresarial usable, acercando la solución técnica al usuario final. De esta manera, se demuestra el valor práctico añadido para el sector aéreo y se facilita su adopción tanto por parte de aerolíneas como de usuarios operativos.

5. Simulación de costes (humanos y técnicos)

La simulación del ROI se realiza tomando como referencia datos económicos oficiales del ejercicio 2025 (Alaska Airlines, 2026).

Datos económicos de Alaska Airlines

- **Total Revenue:** \$11.7 billion.
- **Net Income:** \$395 million (GAAP).
- **Pretax Margin:** 4.6% (GAAP), or 7.1% on an adjusted basis.
- **Growth:** Revenue grew by 12.56% from 2023 to 2024

Alaska Air Annual Revenue (Millions of US \$)		Alaska Air Quarterly Revenue (Millions of US \$)	
2025	\$14,239	2025-12-31	\$3,632
2024	\$11,735	2025-09-30	\$3,766
2023	\$10,426	2025-06-30	\$3,704
2022	\$9,646	2025-03-31	\$3,137
2021	\$6,176	2024-12-31	\$3,534
2020	\$3,566	2024-09-30	\$3,072
2019	\$8,781	2024-06-30	\$2,897
2018	\$8,264	2024-03-31	\$2,232
2017	\$7,894	2023-12-31	\$2,553
2016	\$5,925	2023-09-30	\$2,839
2015	\$5,598	2023-06-30	\$2,838
2014	\$5,368	2023-03-31	\$2,196
2013	\$5,156	2022-12-31	\$2,479

Nota. Datos de ingresos históricos de Alaska Airlines obtenidos de Macrotrends (s.f.).
<https://www.macrotrends.net/stocks/charts/ALK/alaska-air/revenue>

Según la hoja informativa de Alaska Air Group (2025), Alaska Airlines Inc. realiza un promedio de 1.500 vuelos diarios y atiende a 55M de pasajeros.

A partir del análisis del dataset seleccionado, se observó que cerca del 1 % de los vuelos llegan con retraso, lo que permite caracterizar la frecuencia de retrasos en la muestra estudiada.

Resultados del ROI:

```

Alaska Airlines Inc.
Retrasos de llegadas >= 180 minutos.
A partir de 3 horas hay penalizaciones.

min >=180 min.... 181.00 min
mean..... 288.12 min
median..... 248.00 min
max..... 883.00 min

Del total..... 0.94 % - Porcentaje de vuelos cuyos retrasos en llegadas son mayores o iguales a 3 horas.
Total de vuelos diarios... 1,500 - dato oficial Alaska Airlines Inc.
Posibles retrasos..... 14 vuelos al día
Ahorro combustible al año: 12,879.60 EUR
Tiempo total por retrasos por día: 288 min
Alaska Airlines Inc. tiene 30.000 personas - dato oficial Alaska Airlines Inc.. Si tienen un ratio de 6 personas por vuelo,
y son 14 vuelos con retraso, estamos hablando que se requieren 84 personas.
Gastos personal horas extraordinarias: 222,653.12 EUR. Asumiendo que el 5.0% de los vuelos con retrasos son esnhorario extraordinario

Total pasajeros por año: 55,000,000 - Dato oficial Alaska Airlines Inc.
Si la cantidad de pasajeros al año es de 55,000,000 y son 1,500 vuelos al día,
entonces Alaska Airlines Inc. atiende a 100 cada día.
Retrasos vuelos (total vuelos 1500 X 0.94%): 14
Precio promedio por ticket: 575 EUR
Total pasajero por día: 100
...
Gastos por penalización (575 EUR X 14 vuelos X 100 pasajeros): 815,297.00 EUR
Asumimos un beneficio del 20% sobre los gastos de penalización: 163,059.40 EUR
Total de beneficios (12,879.60 + 222,653.12 + 163,059.40): 398,592.12 EUR
ROI 39.53%

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

Se filtra por 180 minutos porque, según la normativa de la **U.S. Department of Transportation (n. f.)**, a partir de 3 horas los usuarios pueden solicitar una recompensa económica a las aerolíneas.

La simulación de costes estima la inversión necesaria para desarrollar, desplegar y mantener la solución, así como el retorno esperado mediante la reducción de retrasos y optimización operativa en aerolíneas.

5.1. Costes de Desarrollo

5.1.1. Recursos Humanos - Equipo de Desarrollo

Rol	Dedicación	Coste/Hora	Horas	Coste Total
Data Scientist (2 personas)	50%	45 €	400	18.000 €
ML Engineer	30%	50 €	120	6.000 €
Lead Developer	40%	55 €	160	8.800 €
Total			680	32.800 €

5.1.2. Infraestructura Técnica - Fase Desarrollo

Recurso	Especificaciones	Coste Mensual	Duración	Total
Servidores Dev	8 CPU, 32GB RAM, 500GB SSD	180 €/mes	4 meses	720 €
Almacenamiento Datos	1TB (dataset 6.7M registros)	40 €/mes	4 meses	160 €
Herramientas Software	Licencias Python/ML	50 €/mes	4 meses	200 €
Total Infraestructura				1.080 €

5.2. Costes de implementación en producción

5.2.1. Desarrollo y despliegue (Meses 5-6)

Actividad	Horas	Coste/Hora	Total
Dockerización & CI/CD	80h	50 €	4.000 €
API Development (FastAPI)	120h	45 €	5.400 €
Testing & QA	60h	40 €	2.400 €
Documentación	40h	35 €	1.400 €
Subtotal	300h		13.200 €

5.2.2. Infraestructura Producción (Anual)

Recurso	Especificación	Coste/Mes	Anual
Servidores Prod	16 CPU, 64GB RAM, Load Balancer	450 €	5.400 €
Base de Datos	PostgreSQL + Redis Cache	200 €	2.400 €
Monitoring	Prometheus + Grafana + Alerting	100 €	1.200 €
Backup & Disaster Recovery	S3 equivalente	80 €	960 €
Total Infraestructura		830 €	9.960 €

5.2.3. Mantenimiento y Operaciones (Anual)

Rol	Dedicación	Coste/Mes	Anual
DevOps Engineer	10% tiempo	500 €	6.000 €
Data Scientist	5% tiempo	225 €	2.700 €
Soporte Técnico	5% tiempo	200 €	2.400 €
Total Mantenimiento		925 €	11.100 €

5.2.4. Imprevistos Primer Año

Fase	Coste	Observaciones
Fase desarrollo (4 meses)	33.800 €	Desarrollo inicial
Puesta en Producción	13.200 €	Meses 5-6
Infraestructura (6 meses)	4.980 €	Mitad año
Mantenimiento (6 meses)	5.550 €	Mitad año
Total	57.530 €	

5.2.5. Inversión Total

Año	Coste
Año 1	125.670 €
Año 2	80.000 €
Año 3	80.000 €
Total	285.670 €

5.3. Beneficios y ROI

5.3.1. Beneficios directos por Reducción de Retrasos

Concepto	Cálculo	Valor Anual
Ahorro Combustible	5 min/vuelo × 14 vuelos/día × 0.5 kg/min × 1€/kg	12.880 €
Optimización Tripulaciones	5% 6p por vuelo x ~14 vuelos son 84 personas/30€h	222.653 €
Reducción Penalizaciones	20% ahorro 575€/ticket/100 pasajeros/14 vuelos	163.059 €
Total		389.592 €

Nota: el mayor ahorro proviene de la optimización de tripulaciones en un 57%

A partir del análisis del dataset, se observa que el 0,94 % de los vuelos presentan retraso. Dado un promedio operativo de 1.500 vuelos diarios, esto implica una estimación de 14 vuelos diarios con llegada tardía.

5.3.2. Análisis de Retorno de la Inversión (ROI) y Payback

Concepto	Valor Estimado
Beneficios directos	389.592 €
Inversión total	285.670 €
ROI anual	39.53 %
Periodo de Repago	9 meses

Nota: La recuperación de la inversión se produce dentro de los primeros 9 meses de operación, lo que posiciona la solución como altamente rentable.

5.4. Plan de implementación

Con el objetivo de garantizar una adopción progresiva, controlada y económicamente sostenible, se propone un despliegue estructurado en tres fases. Este enfoque permite validar el impacto operativo antes de escalar la solución a producción completa.

Fase 1: MVP (Mes 1-4)

- **Objetivo:** Construcción, entrenamiento y validación técnica de los modelos predictivos.
- **Presupuesto:** 45.670 €
- **Entregables:** Modelos entrenados + API básica + Dashboard
- **KPI:** MAE < 7 min | $R^2 > 0.95$

Fase 2: Producción (Mes 5-6)

- **Objetivo:** Industrialización de la solución e integración con entorno operativo.
- **Presupuesto:** 40.000 €
- **Entregables:** Sistema en producción + integración
- **KPI:** Up-time $\geq 99.5\%$ | Respuesta $\leq 200\text{ms}$

Fase 3: Optimización (Mes 7-12)

- **Objetivo:** Mejora continua y expansión funcional.
- **Presupuesto:** 40.000 €/año
- **Entregables:** Mejoras continuas + nuevas features
- **KPI:** ROI $\geq 22\%$

5.5. Conclusiones

El análisis realizado confirma que la solución propuesta presenta una elevada viabilidad operativa y financiera. Con una inversión inicial cercana a 300.000 €, el proyecto alcanza un ROI del **39.53%** en el 1er año.

Los beneficios identificados, especialmente en la optimización de tripulaciones y recursos operativos, demuestran que la aplicación de modelos predictivos de Machine Learning puede generar mejoras tangibles en la eficiencia del sector aeronáutico. La arquitectura modular basada en infraestructura cloud permite además una escalabilidad progresiva, manteniendo economías de escala y sostenibilidad a medio y largo plazo.

Se recomienda iniciar la implementación mediante un enfoque MVP, seguido de una fase piloto con una aerolínea que permita validar resultados en entorno real. La adopción de un modelo de prestación tipo SaaS favorecería ingresos recurrentes y expansión internacional. Asimismo, resulta clave destinar recursos a la mejora continua del modelo y al control del model drift para preservar la precisión y la ventaja competitiva.

6. Manual de usuario

6.1. Objetivo

La herramienta permite predecir retrasos en vuelos comerciales con precisión en minutos mediante modelos de inteligencia artificial entrenados con datos históricos reales. El sistema proporciona estimaciones tanto para el retraso en salida como en llegada.

6.2. Acceso al sistema

La aplicación se ejecuta a través de una interfaz web desarrollada con Gradio, accesible desde navegador en entorno local o desplegada en servidor cloud.

* Running on local URL: <http://127.0.0.1:7860>

* Running on public URL: <https://825718b29861e1360a.gradio.live> (se genera uno nuevo cada vez que se ejecuta el código).

6.3. Procedimiento de uso

6.3.1. Completar Datos del Vuelo

El usuario debe completar los siguientes campos:

- **Detalles del Vuelo:**
 - **Airline:** Seleccionar aerolínea. (ej: American Airlines)
 - **Departure Airport:** Aeropuerto de origen (filtrado automáticamente según aerolínea).
 - **Arrival Airport:** Aeropuerto de destino (filtrado según rutas reales).
 - **Day of Week:** Día de la semana.
- **Detalles del avión:**
 - **Manufacturer:** Fabricante del avión.
 - **Aircraft Model:** Modelo específico (filtrado según fabricante).

6.3.2. Realizar predicción

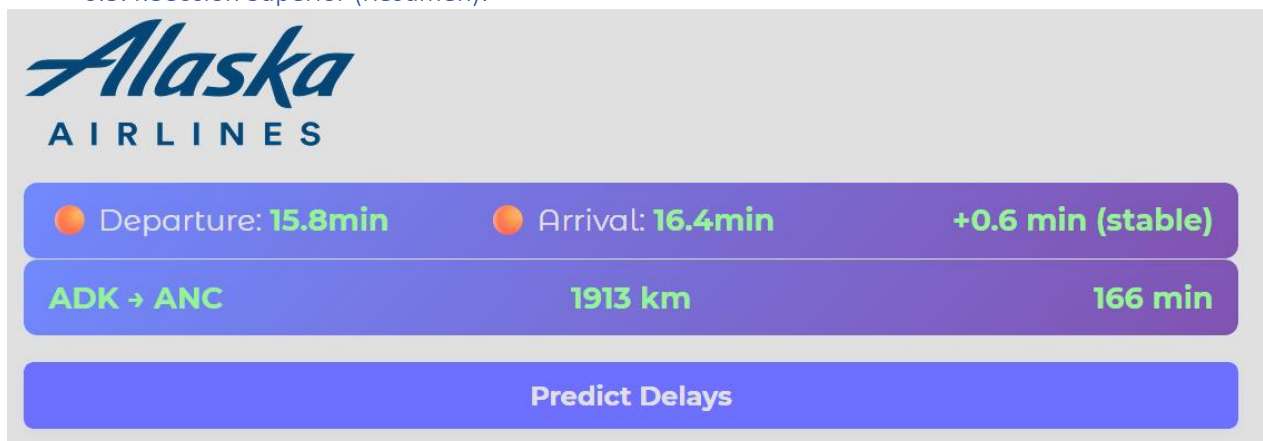
Una vez completados los campos, se debe pulsar el botón de predicción. El sistema procesará la información y mostrará los resultados.

6.3.3. Interpretación de los resultados

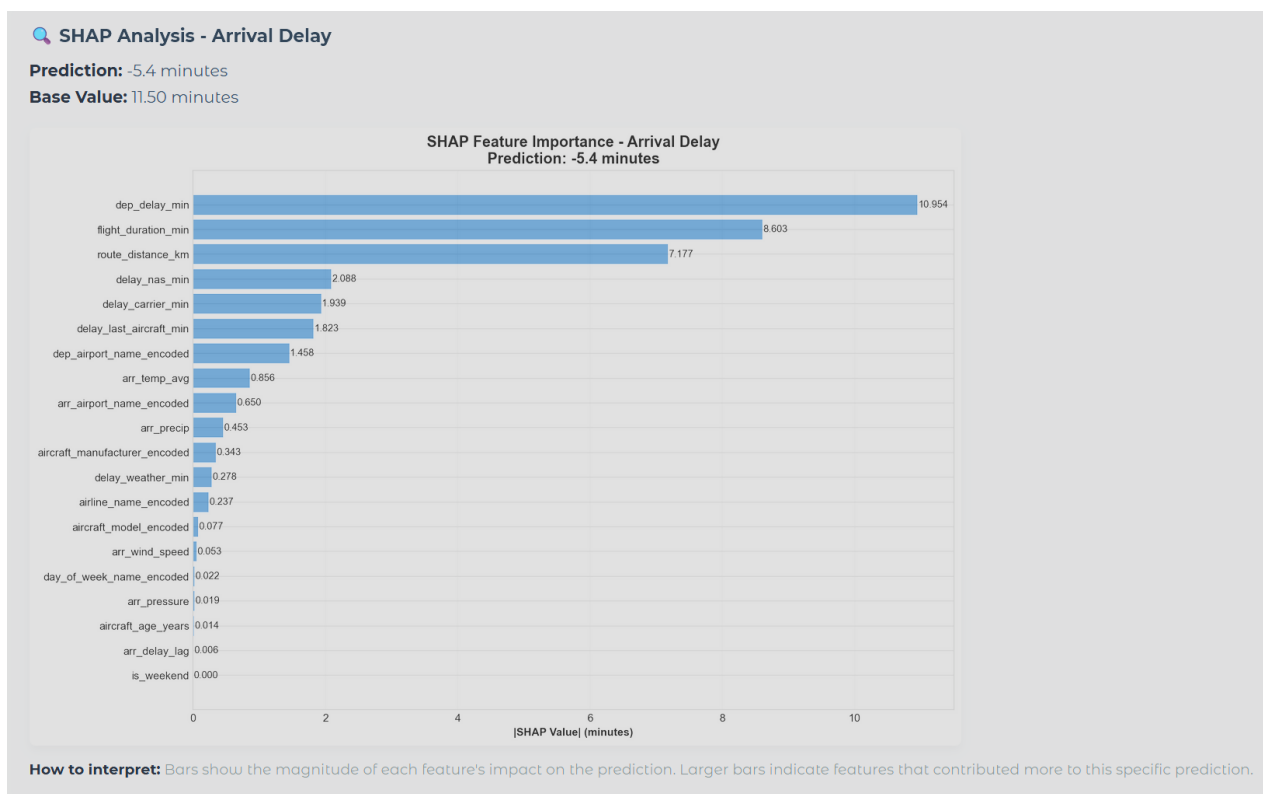
El sistema presenta:

- Retraso estimado en salida (minutos).
- Retraso estimado en llegada (minutos).
- Diferencia temporal entre ambos.
- Clasificación cualitativa del retraso (bajo, moderado o alto).

6.3.4. Sección Superior (Resumen):



6.3.5. Sección Inferior (Detallada):



6.4. Funcionalidades adicionales

- Filtrado dinámico en tiempo real según selección de aerolínea y fabricante.
- Ejemplos precargados para pruebas rápidas.
- Validación automática de campos obligatorios.

6.5. Consideraciones

- El sistema es predictivo y no garantiza resultados determinísticos.
- La precisión depende de la calidad de los datos históricos.
- Se recomienda actualización periódica de los modelos para mantener su rendimiento.

6.6. Resolución de incidencias comunes

Problema	Solución
"No hay aeropuertos disponibles"	Cambiar aerolínea primero
"No hay modelos de avión"	Verificar fabricante seleccionado
"Predicción no funciona"	Verificar que todos los campos están completos
"Error de conexión"	Verificar que el servidor está ejecutándose

6.7. Buenas practicas

1. Siempre selecciona la aerolínea primero, luego origen y destino.
2. Usa ejemplos pre-cargados para pruebas rápidas
3. Considera también el día de la semana

6.8. Soporte

- Equipo Técnico: obstfmgrupo1@gmail.com
- Horario: Lunes a Viernes, 9:00-18:00
- Respuesta: < 24 horas para consultas técnicas

7. Conclusiones

En este trabajo se analizó el uso de modelos de aprendizaje automático para la predicción de retrasos en vuelos, utilizando variables operativas y técnicas propias del transporte aéreo. El estudio se centró en dos objetivos distintos: el retraso de salida y el retraso de llegada. Aunque ambos están relacionados, se decidió tratarlos por separado, ya que responden a procesos diferentes dentro de la operación de un vuelo. Es importante mencionar que el modelo considera el retraso en las salidas como parte del cálculo para el retraso en las llegadas.

Para abordar el problema, se optó por entrenar y comparar una amplia variedad de modelos de regresión. Se incluyeron modelos lineales, modelos basados en árboles, métodos de boosting, un modelo basado en similitud (KNN) y una red neuronal. Esta comparación permitió observar no solo cuál modelo obtenía mejores resultados, sino también cómo influye la complejidad del modelo en la capacidad de capturar los patrones presentes en los datos.

La evaluación se realizó utilizando métricas complementarias. El MAE permitió medir el error promedio de las predicciones en minutos, lo cual resulta especialmente útil desde un punto de vista operativo, ya que indica cuántos minutos, en promedio, se equivoca el modelo. El RMSE se utilizó para analizar la presencia de errores grandes, penalizando desviaciones importantes entre el valor real y el predicho. Por último, el coeficiente R^2 permitió evaluar qué tan bien cada modelo explica la variabilidad total de los retrasos. Además, se tuvo en cuenta el tiempo de entrenamiento como un criterio adicional para evaluar la viabilidad pensando en una futura puesta en producción del mismo.

Los resultados generales muestran que el problema está correctamente definido y presenta una alta capacidad explicativa. La mayoría de los modelos alcanzaron valores de R^2 elevados, cercanos a 0.97–0.98, lo que indica que las variables utilizadas contienen información suficiente para explicar el comportamiento de los retrasos.

Los modelos lineales lograron un desempeño aceptable, con errores absolutos medios (MAE) en el rango de 6 a 7 minutos. Esto sugiere que existe una relación aproximadamente lineal entre algunas variables y el retraso. Sin embargo, los modelos no lineales consiguieron mejorar estos resultados, evidenciando la presencia de interacciones más complejas entre variables que no pueden ser capturadas completamente por enfoques lineales.

Si bien la diferencia en términos de MAE entre modelos lineales y, LightGBM (aproximadamente 5 minutos frente a 6–7 minutos) no resulta extremadamente amplia, el análisis del RMSE permite observar una brecha más significativa. Dado que el RMSE penaliza en mayor medida los errores grandes, resulta particularmente relevante en este contexto operativo, donde los retrasos extremos tienen mayor impacto en la planificación y asignación de recursos.

En este sentido, LightGBM alcanza un RMSE de 7.87 minutos, mientras que los modelos lineales se sitúan en un rango aproximado de 9 a 10 minutos. Esta diferencia evidencia una mejora sustancial en la capacidad del modelo para reducir errores de gran magnitud. Dado que el RMSE penaliza con mayor peso los errores extremos, esta reducción resulta especialmente relevante en un problema como la predicción de retrasos, donde desviaciones elevadas pueden generar impactos operativos significativos.

De forma consistente, LightGBM se posiciona como el mejor modelo en términos globales de desempeño predictivo para el retraso de salida (dep_delay_min), alcanzando un MAE de 5 minutos, un RMSE de 7.87

minutos y un R^2 de 0.979. Estos resultados lo sitúan por encima del resto de los algoritmos evaluados en las principales métricas de precisión.

En cuanto a la eficiencia computacional, CatBoost presenta un tiempo de predicción inferior, lo que lo convierte en una alternativa competitiva en escenarios de alta demanda de inferencia. No obstante, LightGBM muestra un tiempo de entrenamiento considerablemente menor, aspecto relevante considerando futuras reentrenamientos, actualizaciones periódicas del modelo o escenarios de escalabilidad. Si bien existe una diferencia en el tiempo de predicción, esta no resulta determinante, ya que LightGBM mantiene un rendimiento adecuado en la fase de inferencia, lo cual fue validado mediante pruebas en la aplicación de testing desarrollada. En consecuencia, al combinar la mejor precisión global con un entrenamiento más eficiente y un tiempo de predicción competitivo, LightGBM se justifica como la alternativa más equilibrada y adecuada para su implementación.

Para la predicción del retraso de llegada (`arr_delay_min`), el comportamiento observado entre los modelos fue similar al de la salida. LightGBM vuelve a destacarse como el mejor modelo, alcanzando un MAE de 3.72 minutos, un RMSE de 5.66 minutos y un R^2 cercano a 0.99, lo que confirma una capacidad explicativa muy elevada. Asimismo, presenta un tiempo de predicción bajo (0.96 segundos), reforzando su viabilidad operativa en entornos reales.

Cabe destacar que el error obtenido en llegada es incluso inferior al observado en salida. Este resultado puede explicarse porque el retraso de llegada incorpora información acumulada durante la operación del vuelo, incluyendo el retraso en la salida y variables operativas adicionales, lo que incrementa la capacidad predictiva del modelo. En otras palabras, el fenómeno presenta mayor estructura y menor incertidumbre residual en comparación con la predicción previa al despegue.

Si bien los valores elevados de R^2 podrían sugerir un posible riesgo de sobreajuste, la estrategia de partición utilizada (70% entrenamiento, 15% validación y 15% test) permitió aislar correctamente la fase de evaluación final. El desempeño observado en el conjunto de test, consistente con los resultados de validación, indica que el modelo generaliza adecuadamente y no presenta evidencias significativas de overfitting. Asimismo, la coherencia de resultados entre distintos algoritmos refuerza la robustez del enfoque adoptado.

Los rankings de modelos obtenidos para ambos objetivos refuerzan estas conclusiones, ya que muestran de manera consistente que los modelos seleccionados se ubican en las primeras posiciones en términos de desempeño global. Esto confirma que la elección final no se basa en un resultado aislado, sino en una evaluación comparativa coherente respaldada por múltiples métricas.

Los rankings de modelos obtenidos para ambos objetivos refuerzan estas conclusiones, ya que muestran de manera consistente que los modelos seleccionados se ubican en las primeras posiciones según las métricas definidas (MAE, RMSE y R^2). Dichos rankings fueron contruidos automáticamente a partir del proceso de evaluación implementado en Python, ordenando los modelos en función de las métricas cuantitativas establecidas. Esto confirma que la elección final no se basa en un resultado aislado, sino en una evaluación estructurada y objetiva, sustentada en múltiples indicadores de desempeño y en un procedimiento sistemático de comparación.

Se reconoce que el uso de Label Encoding puede afectar el desempeño de modelos lineales y basados en distancia como KNN, dado que introduce relaciones ordinales artificiales entre categorías nominales. No obstante, esta técnica resulta adecuada y eficiente para modelos de boosting basados en árboles, como LightGBM y XGBoost, los cuales demostraron el mejor equilibrio entre capacidad predictiva y eficiencia computacional en este estudio.

En síntesis, el estudio demuestra que la selección del modelo óptimo no debe basarse exclusivamente en la métrica de error más baja, sino en un análisis balanceado que considere precisión, costo computacional, escalabilidad y viabilidad operativa. Este enfoque integral permite trasladar el resultado técnico a un escenario real de aplicación, maximizando el valor práctico del modelo desarrollado.

Adicionalmente, el proyecto avanzó hacia la construcción de un prototipo funcional desarrollado en Gradio, que permite visualizar y utilizar las predicciones de manera interactiva. Esta implementación constituye un puente entre el modelo técnico y su aplicación práctica. Desde la perspectiva académica, permite demostrar de forma tangible el desempeño del sistema y validar su funcionamiento con casos reales. Desde el punto de vista del usuario final, elimina la necesidad de conocimientos técnicos, facilitando la adopción del modelo mediante una interfaz clara e intuitiva. A nivel de desarrollo, la arquitectura implementada resulta fácilmente extensible, permitiendo integración con APIs, incorporación de nuevas variables y pruebas comparativas entre modelos (A/B testing), sentando las bases para una futura puesta en producción.

Además, tanto el modelo como la interfaz incorporan el análisis SHAP, lo que permite interpretar la contribución individual de cada variable en la predicción. En el modelo de retraso en salida, las variables más influyentes corresponden a los retrasos atribuibles a la aerolínea (`delay_carrier_min`) y al retraso del vuelo previo de la misma aeronave (`delay_last_aircraft_min`), confirmando la relevancia de los factores operativos internos y la propagación estructural de demoras. En el caso del modelo de predicción de retraso en llegada, la variable con mayor influencia es el retraso en salida (`dep_delay_min`), lo que evidencia el fuerte efecto de propagación entre fases consecutivas del vuelo. También presentan alta relevancia la duración del vuelo y la distancia de la ruta, reforzando la importancia de las características operativas y estructurales del trayecto.

Por último, el análisis económico desarrollado permitió evaluar la viabilidad financiera del proyecto. Los resultados obtenidos muestran un período de repago en 9 meses, con un ROI anual estimado en aproximadamente 39.53 %. Estos indicadores refuerzan no solo la viabilidad técnica del modelo, sino también su atractivo estratégico y financiero, demostrando que pequeñas mejoras en la anticipación de retrasos pueden traducirse en impactos económicos altamente significativos.

En conclusión, el estudio demuestra que la selección del modelo óptimo debe basarse en un análisis equilibrado que considere precisión predictiva, eficiencia computacional, escalabilidad, viabilidad operativa e impacto económico.

8. Próximos pasos

Como próximos pasos, se propone realizar una prueba en un entorno real con el objetivo de medir el impacto operativo del modelo en la toma de decisiones. Esta validación permitiría evaluar su desempeño en condiciones dinámicas, así como cuantificar su aporte en términos de mejora en puntualidad, eficiencia operativa y reducción de costos.

Adicionalmente, se plantea mejorar las variables utilizadas. En particular, podría incorporarse información meteorológica en tiempo real mediante la integración del sistema con una API especializada en datos climáticos. Asimismo, se prevé la integración con sistemas de seguimiento de vuelos en tiempo real para incorporar variables como el retraso del vuelo previo de la misma aeronave (`dep_delay_lag`). La inclusión de esta información permitiría modelar con mayor precisión los efectos encadenados entre vuelos consecutivos y reforzar la capacidad predictiva del sistema en entornos operativos reales.

Por otra parte, se considera fundamental actualizar la base de datos de tráfico aéreo, actualmente correspondiente al año 2023. Dado que el transporte aéreo es un entorno altamente dinámico, contar con datos más recientes permitiría reentrenar el modelo con información representativa del contexto actual, mejorando su robustez y capacidad de generalización.

En futuras iteraciones, también se propone realizar una búsqueda de hiperparámetros más exhaustiva, así como aplicar técnicas de validación más avanzadas, con el fin de optimizar aún más el desempeño del modelo y reforzar su estabilidad ante distintos escenarios temporales.

Desde el punto de vista tecnológico, el prototipo desarrollado en Gradio puede evolucionar hacia una arquitectura más orientada a producción, incluyendo:

- Implementación como API REST.
- Integración con sistemas operativos de planificación.
- Automatización del pipeline de entrenamiento y despliegue (MLOps).
- Implementación sobre infraestructura cloud escalable.

Esta transición permitiría transformar el prototipo actual en una solución productiva completamente integrada dentro del ecosistema tecnológico de la organización.

En conjunto, estos próximos pasos permitirían evolucionar el proyecto desde una solución predictiva hacia un sistema inteligente de apoyo a la decisión, integrando analítica avanzada dentro del núcleo operativo del negocio y generando ventajas competitivas sostenibles.

9. Bibliografía

1. Bordanova, A. (2023). 2023 US civil flights delay, meteo and aircraft. Kaggle. <https://www.kaggle.com/datasets/bordanova/2023-us-civil-flights-delay-meteo-and-aircraft>
2. Melgosa García, R. (s.f.). Fundamentos de Machine Learning: Apuntes teóricos IA.
3. AlBassam, S. A. A., & AlShahrani, D. N. (2025). Flight delay prediction: Evaluating machine learning algorithms for enhanced accuracy. PLOS ONE, 20(12), e0335141. <https://doi.org/10.1371/journal.pone.0335141>
4. IEEE. (2023). Prediction of flight arrival delay time using U.S. Bureau of Transportation Statistics (Document No. 10371912). IEEE Xplore. <https://ieeexplore.ieee.org/document/10371912>
5. IEEE. (2024). *Flight delay prediction using advanced machine learning techniques* (Document No. 11267898). IEEE Xplore. <https://ieeexplore.ieee.org/document/11267898>
6. Zhou, H., Li, W., Jiang, Z., Cai, F., & Xue, Y. (2022). *Flight departure time prediction based on deep learning*. Aerospace, 9(7), 394. <https://doi.org/10.3390/aerospace9070394>
7. Alaska Airlines. (2026, 22 de enero). *Alaska Air Group reports fourth quarter and full year 2025 results*. Alaska Air Group. <https://news.alaskaair.com/releases/alaska-air-group-reports-fourth-quarter-and-full-year-2025-results/>
8. Alaska Air Group. (2025). *Alaska Air Group fact sheet*. Alaska Air Group. <https://news.alaskaair.com/alaska-air-group-fact-sheet/>
9. Macrotrends. (s. f.). *Alaska Airlines revenue 2007-2025*. Macrotrends. <https://www.macrotrends.net/stocks/charts/ALK/alaska-air/revenue>
10. U.S. Department of Transportation. (n. f.). *Flight delays*. Air Consumer. <https://www.transportation.gov/airconsumer/flight-delays>